

Review

A Comprehensive Review and Assessment of Cybersecurity Vulnerability Detection Methodologies

Khalid Bennouk ^{1,*}, Nawal Ait Aali ^{1,2}, Younès El Bouzekri El Idrissi ¹, Bechir Sebai ^{3,4},
Abou Zakaria Faroukhi ¹ and Dorra Mahouachi ³

¹ Engineering Sciences Laboratory, National School of Applied Sciences, Ibn Tofail University, Kenitra 14000, Morocco; n.aitaali@um5r.ac.ma (N.A.A.); y.elbouzekri@uit.ac.ma (Y.E.B.E.I.); abouzakaria.faroukhi@uit.ac.ma (A.Z.F.)

² Laboratory of Economic Analysis and Modelling, Faculty of Law, Economic and Social Sciences Souissi, Mohammed V University, Rabat 12000, Morocco

³ ACG Cybersecurity Head Office, 3 Soufflot Street, Cabinet PCH, 75005 Paris, France; bechir.sebai@acgcybersecurity.fr (B.S.); dorra.mahouachi@acgcybersecurity.fr (D.M.)

⁴ Laboratory of ACG Cybersecurity, Campus Cyber, 5-7 Bellini Street, Puteaux, 92800 Paris, France

* Correspondence: khalid.bennouk@uit.ac.ma

Abstract: The number of new vulnerabilities continues to rise significantly each year. Simultaneously, vulnerability databases have challenges in promptly sharing new security events with enough information to improve protections against emerging cyberattack vectors and possible exploits. In this context, several organizations adopt strategies to protect their data, technologies, and infrastructures from cyberattacks by implementing anticipatory and proactive approaches to their system security activities. To this end, vulnerability management systems play a crucial role in mitigating the impact of cyberattacks by identifying potential vulnerabilities within an organization and alerting cyber teams. However, the effectiveness of these systems, which employ multiple methods and techniques to identify weaknesses, relies heavily on the accuracy of published security events. For this reason, we introduce a discussion concerning existing vulnerability detection methods through an in-depth literature study of several research papers. Based on the results, this paper points out some issues related to vulnerability databases handling that impact the effectiveness of certain vulnerability identification methods. Furthermore, after summarizing the existing methodologies, this study classifies them into four approaches and discusses the challenges, findings, and potential research directions.

Keywords: vulnerability detection; CPE; CVE; CWE; AI model; graph representation; feature model; similarity matching algorithm; VMS; cybersecurity



Citation: Bennouk, K.; Ait Aali, N.; El Bouzekri El Idrissi, Y.; Sebai, B.; Faroukhi, A.Z.; Mahouachi, D. A Comprehensive Review and Assessment of Cybersecurity Vulnerability Detection Methodologies. *J. Cybersecur. Priv.* **2024**, *4*, 853–908. <https://doi.org/10.3390/jcp4040040>

Received: 1 August 2024

Revised: 22 September 2024

Accepted: 26 September 2024

Published: 7 October 2024



Copyright: © 2024 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

In the first half of 2024, a noticeable increase in cyberattacks and costs associated with managing cyber threats was observed [1]. Gartner declared that business investment in information system security reached more than USD 188 billion in 2023 [2]. Moreover, as cyberattacks become more complex and system configurations vary further, cybersecurity experts continue to work to maintain a logical balance between Confidentiality, Integrity, and Availability (CIA) by making targeted systems more resilient to cyber risks. This situation requires concrete actions to continuously monitor and control the state of hyper-connected systems, providing a comprehensive overview of their security level. To be more efficient, many businesses and corporations deploy different categories of cybersecurity solutions without apprehending their methodologies and techniques, which are concealed and entrenched in the background. Additionally, the European Union Agency for Cybersecurity (ENISA) published in their report a new study, which focuses on threats, trends, and scenarios [3]. The results output new concerns and prioritizations in the cybersecurity field.

In contrast, the adoption rate of solutions for anticipating and reducing cyber risks remains insufficient. This is concerning as the frequency and complexity of cyberattacks increase proportionally with the growth of digital transformation and Industry 4.0 in both IT and OT ecosystems [4]. In this context, and for cyber experts, it is legitimate to define an accurate context in terms of assets control; this step is very crucial during the risk assessment process and constitutes the cornerstone of cyberattacks detection, prediction, and anticipation.

In general, to acquire a realistic picture of an organization's system configuration, a vulnerabilities management system (VMS) can be implemented to supervise and monitor the system state and consequently minimize potential damage from cyberattacks. These systems are regarded as a strategy that contributes to human efforts for detecting faults or vulnerabilities in an organization's information system, internal controls, or system operations. Based on the asset mapping process [5], the VMS discovers potential cyber risks by detecting, assessing, and rating the magnitude of vulnerabilities that might impact software, hardware products, Operator Systems (OS), and Operational Technologies (OT) [6]. More specifically, the majority of VMS performs their aims through four broad phases: inspection and scanning, vulnerability identification, analysis, and reporting. Furthermore, the VMS has to be linked to Vulnerability Databases (VDBs) so that it may be fed with the most recent vulnerabilities and complementary metadata. This step remains so crucial in determining a system for patching priority process. In this field, the handling of vulnerability activities and system configuration is a complex process that involve two essential features: CVE (Common Vulnerability and Exposures) feeds and CPE (Common Platform Enumeration). CVE is a part of the SCAP specification [7]; it represents a method for assigning identifiers to the publicly known vulnerabilities and providing information about the vulnerabilities, whereas CPE specifies a naming scheme for applications, hardware devices, and operating systems [8].

In this context, fully automated vulnerability analysis refers to the capability of VMS to assign a CPE identifier to a configuration product and extracted information (CPE entries) from multiple open VDBs (CVE feeds) in order to perform a series of scans related to potential vulnerabilities without human interaction. Unfortunately, this operation follow a complex procedure which outputs globally a significant rate of false positives or negatives and is qualified as impractical and error-prone [9]. Actually, the wide range of configuration systems increases the workload of security analysts, making it both time-consuming and error-prone when handled manually. The aforementioned difficulties in this context refer to CVE feeds without CPE entries, software products without assigned CPE, the CPE dictionary deprecation issues, or VDBs synchronization between the CPE dictionary and CVE feeds [5,10]. Another issue is the inconsistency challenge of program names across multiple VDBs [11]. It is worth noting that a fully automated CPE assignment is prone to errors owing to CPE and CVE shortcomings (inconsistencies in VDBs and software naming specification difficulties). As a result, the mismatching and inconsistencies might have serious consequences related to dissemination of inaccurate vulnerability information. In this study, we attempt to highlight the existing methods incorporated by various VMSs that enable the matching process between the asset mapping of an Information System (IS) and multiple VDBs since 2016. We also examine the methodology of each approach and provide suggestions for future work. The main contributions of this paper are summarized as follows:

- Conduct a security vulnerability database study to assess data inconsistency and identify issues;
- Classify and analyze vulnerability detection methods according to multiple approaches;
- Provide details of presentation and comprehensive analysis of the drawbacks and limitations of existing vulnerability detection methods in each approach;
- Categorize existing vulnerability detection methods by approaches based on related papers.

The aforementioned contributions will be guided by the following research questions:

1. What are the main methods used in vulnerability detection?
2. How do these methods accomplish their goals and what are their limits?
3. Is it feasible to combine multiple methods simultaneously to reduce the rate of false positives and negatives in the vulnerability detection process?

In this sense, our paper is organized as follows; in Section 2, we present the research methodology. Then, we discuss the motivation and background in Section 3. After that, Section 4 describes the extent of related studies, including further detail on the existing main approaches related to the vulnerability detection field. Section 5 highlights challenges and potential solutions for vulnerability detection methods. Finally, Section 6 concludes with findings and discusses future research.

2. Research Methodology

The methodology adopted followed a systematic literature review (SLR), proposed by authors in [12], to derive conclusions and reflections about the above research questions. This academic approach helped us gather, examine, sort, and study the pertinent papers within the topic frame. The recommended guidelines of this method consist of three main stages:

- Planning the review, which focuses first on the identification of the need for a review, their proposal, and the development of their protocol;
- Conducting the review involves identifying the research using predefined keywords and search strings, selecting the studies based on inclusion and exclusion criteria, performing a study quality assessment using predefined criteria and checklists, extracting data, and monitoring progress before summarizing findings and providing data synthesis;
- Reporting recommendations and disseminating evidence through a descriptive analysis of findings and insights.

Consulting several reputable academic libraries helped us to gather pertinent articles related to our subject and respond to the research questions. These libraries are as follows:

1. ACM (Association for Computing Machinery) digital library;
2. JSTOR;
3. IEEE Xplore digital library;
4. MDPI;
5. ScienceDirect;
6. Scopus;
7. Springer;
8. Web of Science.

The current study aims to collect pertinent papers published from 2016 to 2024. To this end, many specific keywords are used in the research methodology during this period, such as: “CPE and CVE”, “vulnerability detection”, “vulnerability assessment”, “CWE and vulnerabilities”, “matching vulnerabilities”, “asset inventory and CPE”, “vulnerability detection and AI”, “CVE and CPE by graph”, “CVE and CPE by FM” and “VMS and vulnerability detection”.

As shown below in Figures 1 and 2, the research method consisted of four procedures to gather the most significant papers related to our subject. The first stage involves gathering and building a global overview of the scientific contributions found in the literature review. Next, this study initially retrieved 846 papers from the academic libraries. By eliminating duplicates and out-of-scope papers, and classifying the publications using the abstract and title, the paper number was reduced to 487 papers. Then, 256 articles were selected by using predetermined criteria relevant to our topic. The following criteria were adopted:

- Papers published within the last 8 years;
- Relevant papers according to the research question posed previously;
- Papers suggesting vulnerability detection methods;

- Methods leveraging the usage of basic security metadata or AI techniques;
- Papers offering well-documented research on the proposed methods.

To provide unbiased research, the analysis was limited to academic contributions focusing on the described methods relative to our topic. Ultimately, data analysis results (125 articles) were separated into two studies: the main study, which conducts a thorough and deep investigation of the article's content, and the connected study, which is sufficiently investigated to derive further insights and future contributions.

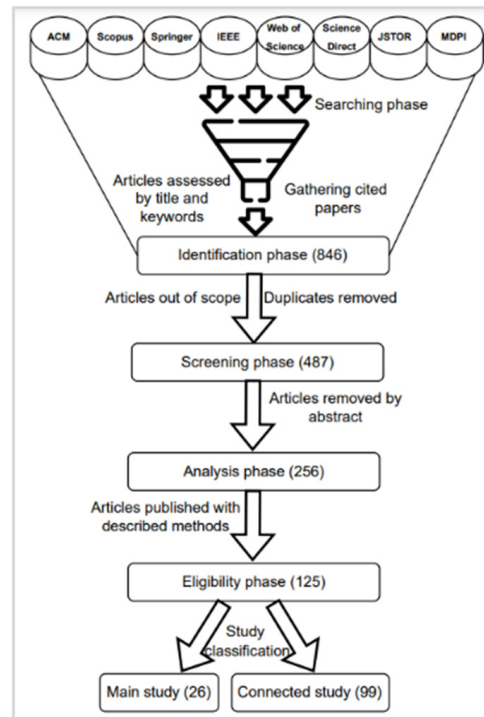


Figure 1. Process of the methodology used in the literature review.

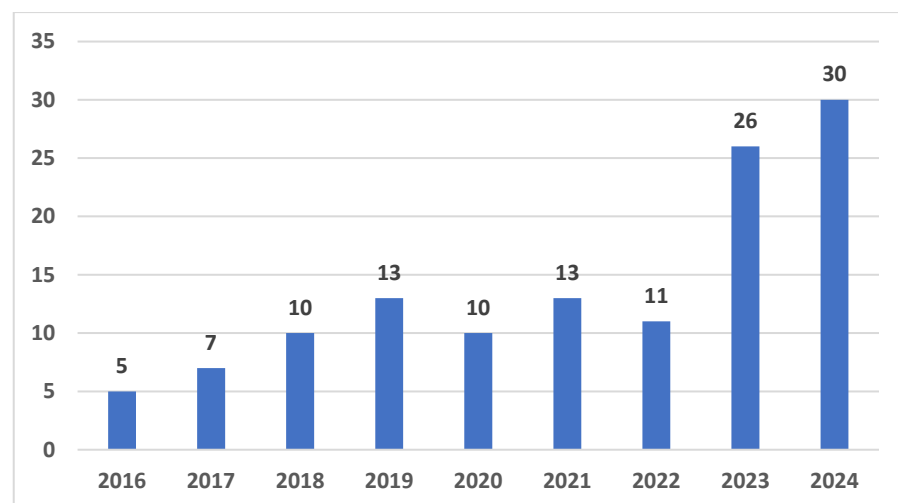


Figure 2. Distribution by year of the analysis study.

Thus, the previously used methodology framed our study to find pertinent papers according to our research topic. In the following section, we will present motivation, some basic cybersecurity concepts, and an overview of security events published in the National Vulnerability Database (NVD).

3. Motivation, Background, and VDB Assessment

This section provides an overview of the global motivation for this work and the technological basis and concepts to easily navigate this paper.

3.1. Motivation

Specifying the precise inventory is so crucial for assessing vulnerabilities. In other words, detecting vulnerabilities that may affect inventory products remains a complicated with a high incidence of false positives and negatives. Meanwhile, this operation requires two vectors, notably the specification of the installed products and their associated vulnerabilities. These relevant data are retrieved from the target system, the cybersecurity event management databases, websites, and other sources. As a result, the mapping process identifies the target products potentially affected by vulnerabilities. Unfortunately, automating this process faces multiple challenges [5,10,11]:

- Various configuration systems impact product inventories and technical content of VDBs;
- Product properties, such as name, version, and edition, might change frequently affecting mapping with VDBs and inventory systems;
- Vulnerability databases that list the same product under different properties have inconsistent product names (character and semantics);
- Inconsistencies in vulnerability databases, including both structured and unstructured product names.
- Relevant insights may reveal CVE feeds without CPE entries;
- Some product vulnerabilities, including software, hardware, and operating systems are published without assigned CPE;
- Product identity is not unified across information systems and VDBs;
- Some CVE feeds contain CPE entries that are not in the CPE dictionary;
- The high rate of false positives and negatives in the vulnerability detection process.

3.2. Terminologies and Theoretical Foundations

To aid navigation, we provide an overview of key terminologies and basic concepts. This section begins with key cyber terminologies and concepts related to vulnerability assessment and cyber risk management.

3.2.1. Cyber Fundamentals

The following cyber terminologies are extracted from guidelines and standards. The foundation of cybersecurity is based on the cyber items listed below and helps readers comprehend the remainder of this paper.

Threat: It is a potential source of harm to a system or organization, impacting assets like information, processes, and systems. Threats can be natural or human-made, accidental or intentional. It is important to note that a threat agent can be an individual or a group that plays a crucial role in carrying out or assisting an attack [13].

Vulnerability: Refers to a flaw in an asset or security measure that can be exploited by one or more threats. Vulnerability assessment (VA) is a continuous activity that involves monitoring and identifying these flaws. It must be carried out by cyber experts using a reliable and resilient system [14].

Risk: In general, risk refers to events, consequences, or both, and specifically, when a threat exploits a vulnerability in an information asset or a collection of assets, causing harm to an organization [14]. In addition, the risk can be identified, analyzed, measured, based on impact and occurrence, and subsequently treated. Following numerous standards, such as ISO/IEC 27005 or the NIST Risk Management Framework (RMF) [15,16], it is recommended that organizations apply a PDCA (Plan-Do-Check-Act) technique for continuous development [17].

PDCA: Plan: Identify and assess cyber threats, then strategically consider appropriate risk-reduction measures. Do: Implement these measures. Check: Conduct a performance

review, and Act: Monitor and enhance the risk treatment plan. In contrast, the NIST SP 800 30 aims to analyze risks using three major steps: S1—Risk assessments look at the risks across all organizational levels, S2—Focus on business processes, considering sales, marketing, or HR (Human Resources) procedures, and S3—Leverage the technological level by integrating applications, systems, and information flows [18].

Impact: Defines the magnitude of the harm that can be expected from unauthorized disclosure, alteration, or destruction of information, and loss of information or system availability [19]. These repercussions can affect confidentiality, integrity, availability, or all three.

Security measures: They encompass any processes, policies, devices, practices, or other activities that may be administrative, technological, managerial, or legal in nature that are meant to change a risk state. Classified by their function, security measures can be preventive, detective, or corrective [20,21].

Exploit: It refers to the frequency of attacks targeting assets, exploiting a specific vulnerability, and the likelihood of a vulnerable system being attacked [22].

Assets: It includes data, personnel, devices, systems, and facilities that enable the organization to achieve business objectives [20]. Assets may be divided into two groups: Physical assets include money, equipment, stocks, and items, as well as network and server infrastructure, etc. Virtual assets include accounts, data, business plans, and reputation, etc.

CIA: Confidentiality (C) ensures information is not made available or disclosed to unauthorized individuals, entities, or processes (authentication, authorization, and access control). Integrity (I) protects the accuracy and completeness of assets (information changed). Availability (A) ensures that assets are accessible and usable on demand by an authorized entity [23].

Attack Vector (AV): Specific path or scenario used by a hacker or malicious actor to exploit vulnerabilities and gain access to a target system [24].

Access Complexity (AC): A metric capturing the actions an attacker must take to evade or bypass security measures to exploit a vulnerability [22,25].

In summary, cyberspace integrates software, internet services, information technologies (IT), telecommunications networks, and technology infrastructures. This virtual environment links all previous cyber items directly or indirectly. As shown in Figure 3, any organization, regardless of size, could be able to possess one or more potential vulnerabilities in their assets that might be exploited by a threat and launch a potential attack. The exploitation of vulnerabilities may turn into a major risk assessed based on their impact and occurrence. The organization rates the risks and vulnerabilities' severity by using a risk assessment in a context-aware manner. It then elaborates a mitigation plan to reduce the risk impact regarding the CIA by implementing the necessary security measures. This step follows a risk management process, as shown below in Figure 4. In addition, residual risks may remain even after applying the necessary safety measures. This fact implies a continuous process of control and supervision to prevent further impacts [26,27]. The result should be communicated for tracking and making timely decisions.

3.2.2. Cyber Concepts

We have focused on the key components of the following cyber concepts to help readers understand the content of this paper.

Cyberattack: Malicious activity aimed at collecting, disrupting, denying, degrading, or destroying information system resources or the information itself [18].

Cyber resilience: The ability to continuously deliver the intended outcome despite adverse cyber events, encompassing the identification, evaluation, treatment, and reporting of system and software vulnerabilities [20].

Cyber threat: Any circumstance or event that has the potential to harm organizational operations, assets, individuals, other organizations, or the nation by gaining unauthorized access, causing destruction, disclosure, modification of information, and/or denial of service [28].

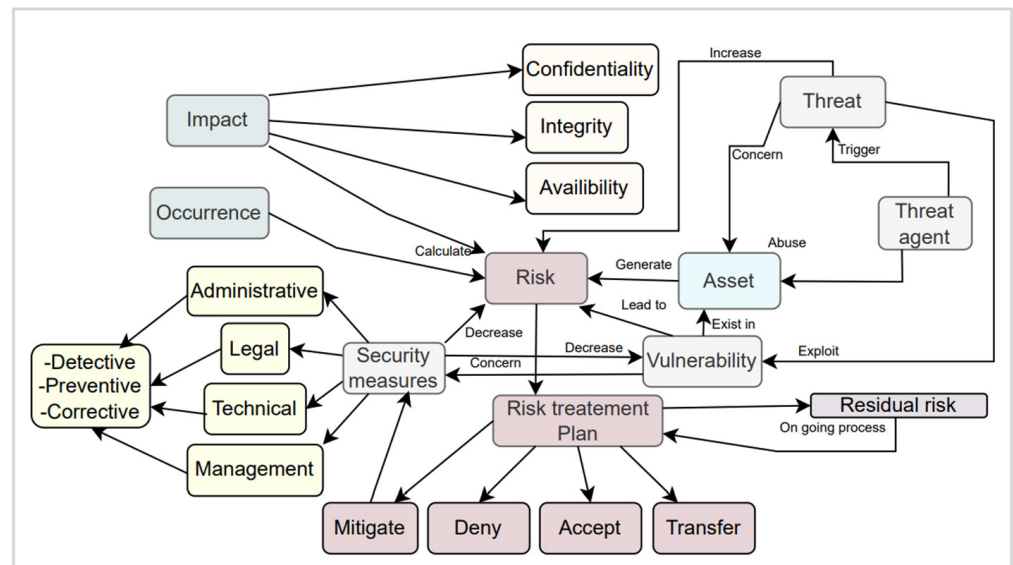


Figure 3. Interaction between information security cyber items.

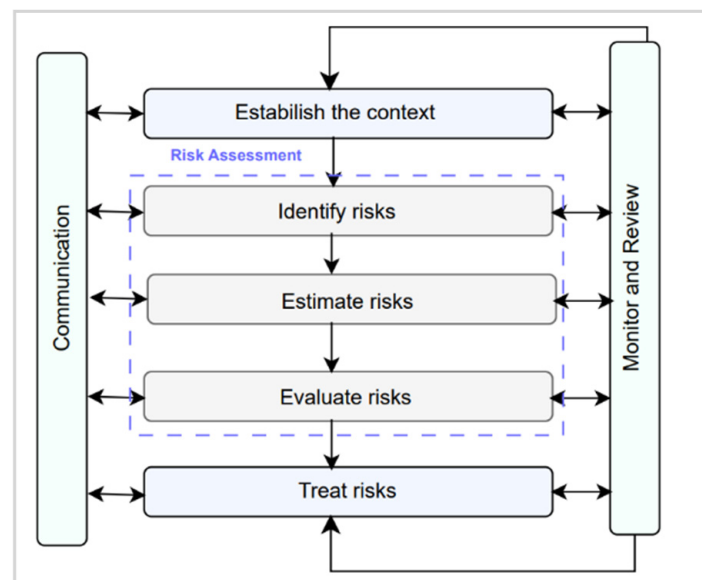


Figure 4. Risk management process [15].

Vulnerability Management System (VMS): It represents a capability that identifies CVEs present on devices that attackers may exploit to compromise them, thereby using them as platforms to further compromise other segments of the network [29]. VMSs incorporate multiple hybrid systems to detect potential cyber risk presence across diverse ecosystems and assess their cyber state.

As shown in Figure 5, the implementation of VMS in different ecosystems (IoT, IT, cloud-based systems, ICS, and others) depends on the quality and quantity of the data gathered from multiple vulnerability databases (VDBs) and the capability to collect information through scanning operations about products affected by known vulnerabilities. In this context, the primary function of the VMS is to perform a logical mapping between CPE/VDBs and product ID, ensuring accurate results while reducing the occurrence of false positives and false negatives [6].

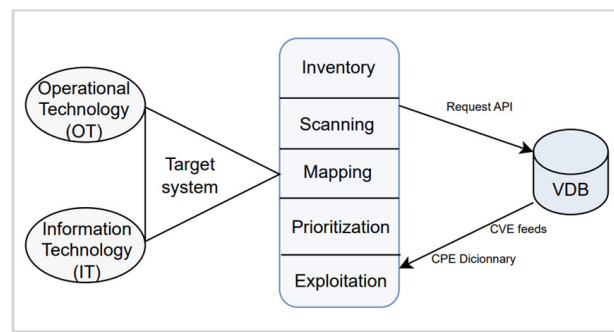


Figure 5. VMS concept.

Common Platform Enumeration (CPE): It represents a structured naming scheme for information technology systems, software, and packages. Based on the generic syntax for Uniform Resource Identifiers (URI), CPE includes a formal naming format, a method for checking names against a system, and a description format for binding text to a name. The CPE dictionary is provided by NIST and is available to the public [30]. The CPE standard was developed to unify product naming; the current version of CPE is 2.3, which is detailed in three representations in Table 1.

Table 1. Diverse representations of CPE.

Format	Description	Representation
WFN	Well-Format Name	$Cpex = \{ \langle part, v1 \rangle, \langle vendor, v2 \rangle, \langle product, v3 \rangle, \dots, \langle other, vn \rangle \}$ $wfn:[part = "a", vendor = "microsoft", product = "internet_explorer", version = "8.0.6001", update = "beta"]$
URI	Uniform Resource Identifiers	$CPE = cpe:/\{part\}:\{vendor\}:\{product\}:\{version\}:\{update\}:\{edition\}:\{language\}.$ $cpe:/a:microsoft:internet_explorer:8.0.6001:beta$
FSB	Format String Binding	$cpe:2.3:part:vendor:product:version:update:edition:language:sw_edition:target_sw:target_hw:other$ $cpe:2.3:a:microsoft:internet_explorer:8.0.6001:beta:*****$

A well-formed CPE name (WFN), an abstract logical construction, refers to this CPE naming method. The CPE naming specification defines procedures for binding WFNs to machine-readable encodings and for reversing these encodings back to WFNs [31]. The CPE standard defines eleven attributes in WFN format. Part (1) may contain “a” for applications, “o” for operating systems, or “h” for hardware devices. The vendor (2) identifies an individual or an organization responsible for producing or developing the item. The official product name is identified by part (3). Version (4), update (5), and sw_edition (6) specify version and update details, with edition (7) typically set to ANY unless backward compatibility requires a specific value related to the product. The user interface language (8) tag follows the RFC 5646 definition [32], while target_sw (9) denotes the product’s operating environment. Target_hw (10) specifies the hardware architecture. Finally, other (11) provides additional information supporting specifications referenced in [8,33].

The public can gain access to CPE information using the NIST API, as shown below in Figures 6–10, which ensures data accuracy, reliability, and accessibility. Figure 6 above presents an example extract from a query. The NIST API is used to preserve and make the CPE data available. CPE can be extracted from CVE/metadata and the NIST/dictionary, as shown below in Figure 11. Figures 9–12 illustrate the annual collection of CPE via Python scripts from 2016 to 2024, highlighting partitions for hardware (h), operating systems (o), and applications (a).

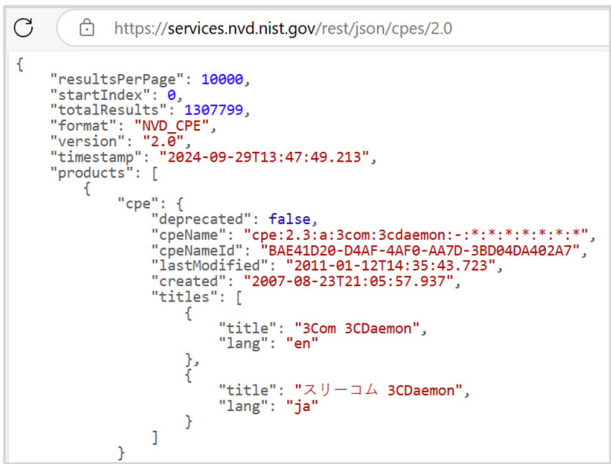


Figure 6. CPE extracted from NVD/NIST API related to “3com” vendor.

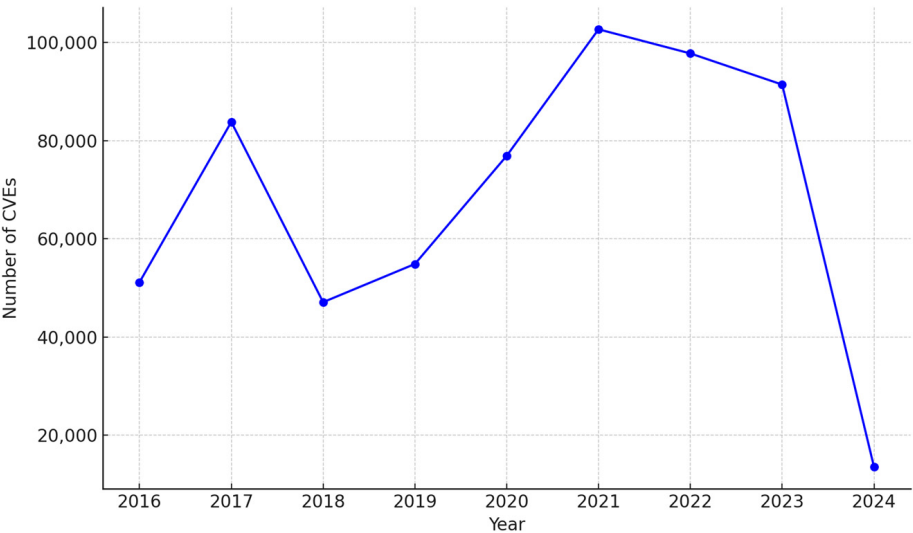


Figure 7. Total of CVEs published by NVD with and without the CPE value.

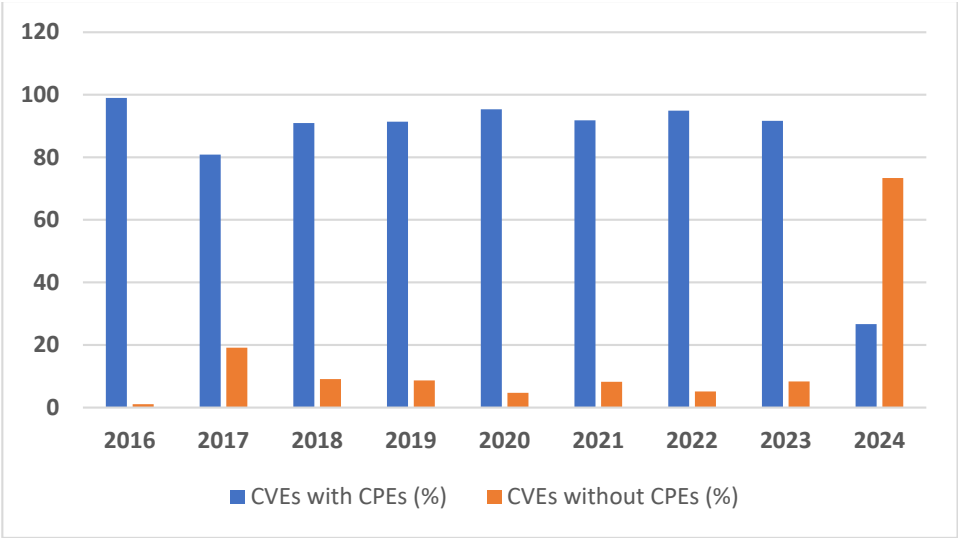


Figure 8. Total of CVE numbers published per year by NVD.

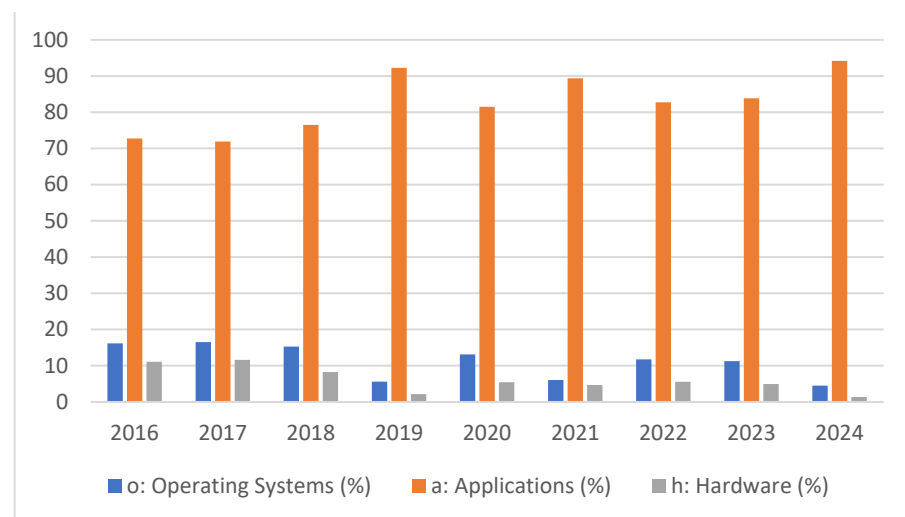


Figure 9. Distribution of CPEs number extracted from NVD/CPE DICT by partition.

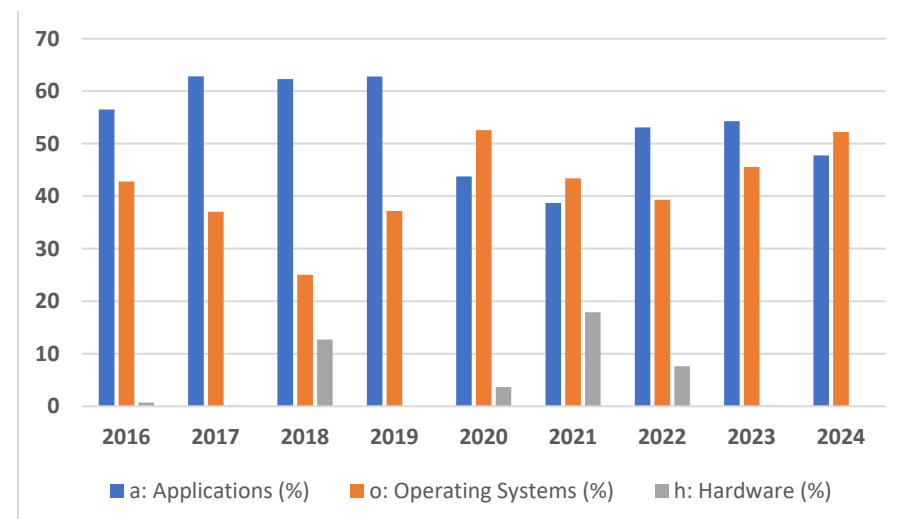


Figure 10. Distribution of CPEs extracted from NVD/CVE API by partition.

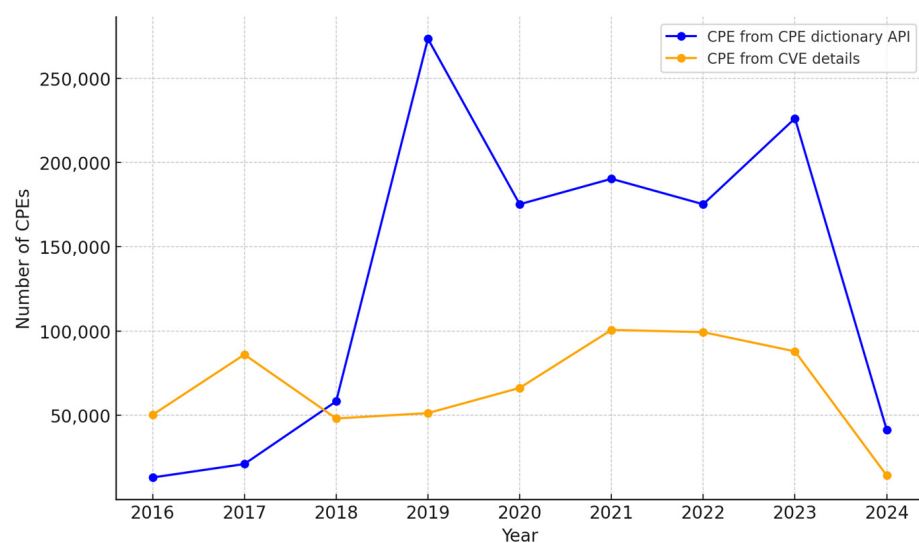


Figure 11. Comparison between CPEs extracted from NVD.

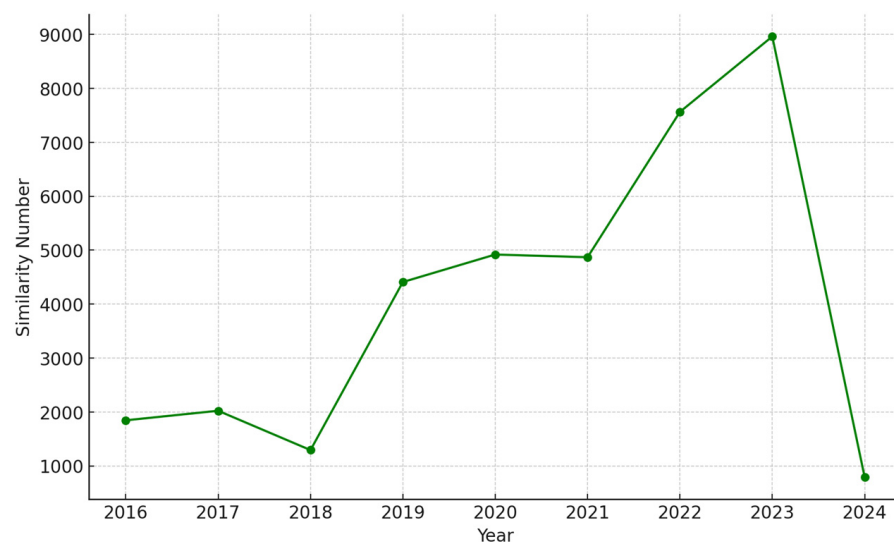


Figure 12. Similarity rate of CPEs between NVD/dictionary and NVD/CVE.

Common Vulnerability and Exposure (CVE): It is a program maintained by the MITRE Corporation [34] and sponsored by the U.S. Department of Homeland Security (DHS) and the Cybersecurity and Infrastructure Security Agency (CISA) [35]. It focuses on representing a nomenclature and dictionary of security-related product flaws. Every CVE ID is assigned to the respective product by authorized organizations known as CVE Numbering Authorities (CNAs). The National Vulnerability Database (NVD) manages the analysis process for each CVE ID, incorporating reference tags, the Common Vulnerability Scoring System (CVSS), the Common Weakness Enumeration (CWE), and CPE Applicability Statements [36]. It is worth noting that the number of CVEs published by the NVD increases annually. Figures 9–12 provide statistics on supplemental CPE information between 2016 and 2024. This highlights a notable discrepancy between CPEs released with CVE/metadata and those listed in the dictionary. Additionally, it is important to recognize that not all CPEs affected by disclosed vulnerabilities are covered in every CVE entry.

Common Weaknesses Enumeration (CWE): It can be understood as a state within a hardware, software, firmware, or service component that, under specific conditions, can lead to vulnerabilities. CWE incorporates a taxonomy to identify common sources of weaknesses [37].

Scoring system: Each year sees an increase in the number of published vulnerabilities, with a notable peak in 2021, as illustrated in Figure 7, while their severity remains influenced by various factors. In this context, employing a scoring system becomes essential to classify complexity and prioritize assessment processes. In our literature review, we identified four distinct scoring systems. The Common Vulnerability Scoring System (CVSS) is the first method used to address the vulnerability impact using qualitative representation (low, medium, high, and critical) and quantitative measures of severity (a scale from 0 to 10).

The last version, CVSS V4.0, was released in November 2023. It adds more information, including significant changes from the previous versions of CVSS V3.x and V2.x, additional scoring guidance, and scoring rubrics [38]. The second system is the Vulnerability Rating and Scoring System (VRSS) [39], which bases its final score on CVSS V2, providing both qualitative ratings and quantitative scores for vulnerabilities. The third system is called the Weighted Impact Vulnerability Scoring System (WIVSS). Based on CVSS V2, it assigns different weights for CIA impact metrics in contrast to CVSS, which uses the same weights for impact metrics [40]. Finally, the Variable Impact–Exploitability Weightage Scoring System (VIEWSS) is a hybrid technique that combines the strengths of CVSS, VRSS, and WIVSS [41].

Incident Response (IR): It is focused on identifying, analyzing, and mitigating damage, as well as addressing the root to minimize incident impact. This can be viewed as the

mitigation process for security violations of policies and recommended practices. Incident Response (IR) encompasses eight broad operations within any ecosystem: policies and procedures (IR), training (IR), testing incidents, handling incidents, monitoring incidents, reporting (IR), assistance, and an IR plan [42,43].

Indicator of Compromise (IoC): After an attack has been executed on a victim system, some digital footprints can be left by hackers. This evidence of a possible attack represents forensic artifacts from intrusions identified at the host or network level within organizational systems. IoCs provide valuable information about compromised systems and can include the creation of registry key values. IoCs for network traffic include Universal Resource Locators or protocol elements that indicate malicious code commands and control servers. The rapid distribution and adoption of IoCs can enhance information security by reducing the time systems and organizations remain vulnerable to the same exploit or attack [43].

Thus, this section highlighted the most important cyber elements and concepts to provide a foundation for understanding the content of this paper. Next, we will examine additional findings regarding used VDBs.

3.3. Security Vulnerability Databases

Several security databases are responsible for publishing vulnerability information, technical details, and other complementary resources. This paper focuses on seven repositories that handle relevant vulnerability information. We also highlight other indicators related to these databases and how these data are presented to the public. The details are summarized below in Table 2. In summary, a few of the studied vulnerability databases propose their API to share security cyber events, new vulnerabilities, CWE, attack patterns, CPE, and other relevant information. If the data are sufficiently complete and accurate, cyber experts can focus on automating, deeply analyzing, and efficiently managing the vulnerability management process.

Moreover, the scoring system focuses on capturing the principal technical characteristics of software, hardware, and firmware vulnerabilities. At this end, upgrading the scoring system to calculate a qualitative severity rating scale for detected vulnerabilities would be beneficial. The CVSS V4.0 [38] includes exploitability and impact metrics, exploit maturity, and environmental metrics. This would assess the dependence of the importance of the affected IT asset, measured in terms of confidentiality, integrity, and availability, and then supplemental metrics, which measure additional extrinsic attributes of a vulnerability and other relevant cyber event information.

Our contribution in this field includes an overview of the NVD data quality assessment concerning published security events from 2016 to 2024. In this context, we designed and developed a Django-based system that maintains a steady connection with the public API and fetching and storing data in a local PostgreSQL database. In addition, the system can execute targeted background updates in our local database in response to NVD changes or updates. Therefore, the fetched data follow a correlation process within the system to prepare a group of data adapted to the context of the ecosystems concerned. Figures 7–12 display the results of the statistical survey conducted based on multiple Python scripts. In this study, we have focused on the quality and quantity of the CVE published to the public, the CPE dictionary, and other security metadata comparisons. The entire statistical analysis presented in this study is based on data provided by ACG Cybersecurity (<https://acgcybersecurity.fr/>, accessed on 2 August 2024). This work will be available on GitHub [44] and will constitute the focus of future research.

Table 2. Security vulnerability databases (VDBs).

VDBs	CVE ¹	NVD ²	Mitre ³	VulDB ⁴	Security DB ⁵	VulnDB ⁶	ExploitDB ⁷
Operated by	Mitre Corp	NIST	Mitre Corp	Scip AG	Varies	Risk-based security	Offensive security
Data delivered	CVE ID Description Severity Product Version	CVE ID Description Metrics CPE References	CVE ID CVE Program	Vulnerabilities technical details Exploit availability, Impact References Product affected	Security research papers Exploit security, Events	vulnerability's technical details Mitigation strategies Exploit information Other Resources	Security vulnerabilities Affected software or technical description of the systems Relevant exploit code
Free access		Yes		Limited free access, Subscription for more information and services (Commercial or Enterprise)	Limited version is free (Just one product is monitored)	No	Yes
Update process		Regularly		Limited for free version	Daily for limited version and hourly for subscriptions		Regularly
API Support	No	Yes	No	Limited for free version	Not available for limited version		Yes
CVE List download		Available		Not available for free version	Not available		Available
Scoring System	CVSS V2, 3 and 4.0		-	CVSS V2, 3.x and 4.0	CVSS V2 and V3	CVSS V2, 3.x and 4.0	CVSS V2 and V3

¹ <https://www.cve.org/>; ² <https://nvd.nist.gov/>; ³ <https://www.mitre.org/>; ⁴ <https://vuldb.com/>; ⁵ <https://www.security-database.com/>; ⁶ https://vuln.db.flashpoint.io/users/sign_in; ⁷ <https://www.exploit-db.com/>. (All accessed on 6 May 2024).

Thus, each of these databases presented above has its own specificities, performance, and accuracy. The analysis of the data published by NVD confirms the existing VDBs' issues and shortcomings, which will be discussed further.

To conclude this section, we presented several motivations for choosing this topic. We focused on key cybersecurity elements, providing concise explanations to facilitate understanding. We also summarized our findings on various VDBs, with an in-depth focus on NVD. In the following section, we will examine our research and explore multiple findings in vulnerability detection.

4. Taxonomy of Vulnerability Detection Approaches and Findings Analysis

Our study focused on thoughts and findings according to vulnerability detection methodologies derived from the literature review, which are then classified into four approaches, as depicted below in Figure 13. The concept of nearly disparate ways of employing the same idea is included in every basket of approaches to identify potentially suspicious security events on products, software, systems, and other devices. It is worth noting that the brute force-based approach is considered out of scope in this study, as it integrates multiple tools and different strategies and is considered time-consuming and resource-intensive. This approach refers to all techniques that systematically attempt to find vulnerabilities by checking every suspect parameter until a vulnerability is discovered [45].

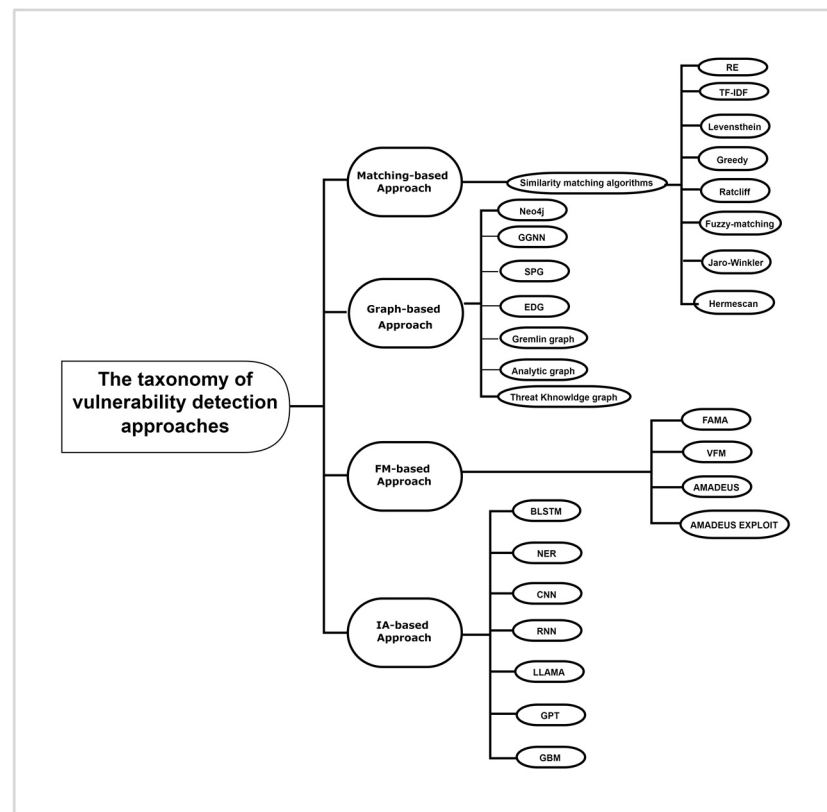


Figure 13. Taxonomy of vulnerability detection.

In this section, we first present the existing methods dedicated to identifying, assessing, and evaluating vulnerabilities found in the literature between 2016 and 2024. Next, we extract the various methodologies employed in the vulnerability mapping process and identify findings analysis, some limitations, and observations. Finally, we highlight a global discussion about the previously mentioned findings.

4.1. Matching-Based Approach

The matching-based approach uses a variety of algorithms, such as Regular Expression, Levenshtein edit distance, Greedy, Jaro–Winkler, Ratcliff/Obershelp, etc., as shown below in Figure 14, to search for vulnerabilities in VDBs using data extracted from the target system. Using multiple scanning modes, these methods of the current approach try to identify suspect flows and lower the false positive or negative rate. The principle consists of matching the CPE of the ID product with the CPE dictionary to identify related vulnerabilities. More details are summarized further through the following literature review.

4.1.1. Matching-Based Approach Methods Description

Method Based on RE

The author presented a method related to vulnerability detection techniques based on file logs rather than active mode, which involves intensive system scanning and missing inactive services [46]. Based on collecting and normalizing system logs, Passive Vulnerability Detection (PVD) uses Regular Expression (RE) to parse and normalize existing information (Unix/syslog, DPKG logs, Windows event data, web server logs, proxies, gateways, etc.). Next, it looks for potential vulnerabilities based on CPE (vendor, name, and version). On the other hand, VDBs (such as HPI-VDB, OSVDB, NVD, etc.) publish recent vulnerabilities with their other metadata, aiding PVD in matching CPE IDs (products and VDBs) to discover the concerning products by any CVE ID.

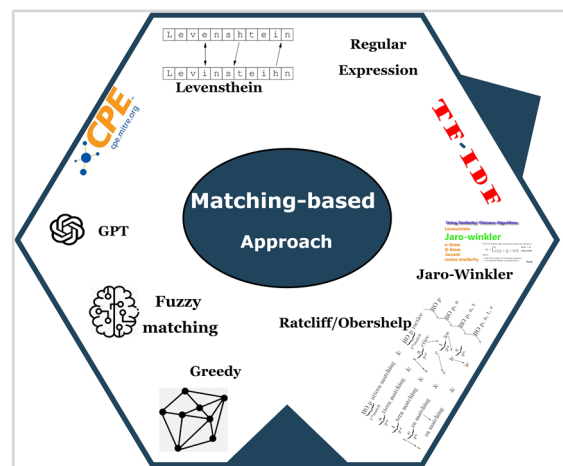


Figure 14. Features of similarity matching-based approach.

Method Based on Levenshtein Algorithm

Another contribution proposed a new technique to detect vulnerabilities within an information system [9]. This work tries to overcome four main issues that make it difficult for a VMS to discover relevant vulnerabilities: (1) lack of synchronization between CPE dictionary and CVE feeds; (2) CVE entries without CPE metadata in VDBs; (3) missing CPE identifiers for certain software products; and (4) deprecation and typographical errors that generate mismatches and a high rate of false positives and negatives. Based on the Well-Formed Name (WFN) of CPE, the exposed method englobes three steps: (S1) CPE matching, which finds the correct CPE for a software product from a CPE dictionary using the Levenshtein distance algorithm, is less or equal to two. Then, (S2) CPE assignment, which is based on human interaction, proposes candidates CPE to select the most similar CPE to the software target, as shown below in Equation (1). Finally, (S3) CVE matching uses the Levenshtein distance algorithm to compare CPE products and CPE/CVE (dictionary and summary description) to find relevant CVEs for a software product.

$$\begin{aligned} &(\text{CPE.WFN.VENDOR} = \text{VENDOR SEARCH TERM}) \text{ AND} \\ &(\text{CPE.WFN.PRODUCT} = \text{PRODUCT SEARCH TERM}). \end{aligned} \quad (1)$$

Method Based on Building CPE

The authors presented another study to automatically generate the correct CPE device by combining the CPE tree generation process and banner text keyword analysis [47]. Subsequently, the generated CPE is then used to identify relevant vulnerabilities from NVD. The method consists first of extracting device information by using specific scanning tools (Nmap and Shodan). Next, based on the tree CPE extracted from the CPE dictionary, it builds the correct CPE for the target device. Finally, it matches device information with CVE feeds. It is worth noting that the study's comparison technique lacked a clear description.

Method Based on TF-IDF

A new method introduced a mechanism based on analyzing vulnerability descriptions at the time of disclosure. This method addresses the problem of sparse or inaccurate metadata at the first appearance of a vulnerability [48]. They use a technique based on TF-IDF weighting of keywords (Term Frequency-Inverse Document Frequency), as shown in Equation (2), to automatically extract relevant keywords from the unstructured, human-readable descriptions, and output the most likely affected software. To increase the relevance of the extracted keywords, additional domain-specific heuristics are used, such

as handling multi-word keywords, capitalized terms, and words starting with “lib-.” Thus, the evaluation is showing a promising result in general.

$$\text{TFIDF}(t,f,D) = \text{TF}(t,d) * \text{IDF}(t,D) \quad (2)$$

where “t” is a word and “d” is a document belonging to a corpus of documents “D”.

Method Based on Binary X-ray

Another contribution proposed a novel solution called BinXray. The principal goal of the method is to differentiate a patched function from a vulnerable program by identifying the integrated patch [49]. To accurately identify 1-day vulnerabilities, BinXray uses the three inputs (see Table 3) to match the target function (TF) and vulnerable function (VF) based on syntactic and structural information. Then, the method extracts the patch signature by computing differences between VF and PF at the basic block level. Next, after generation traces from TF, VF, and PF, BinXray computes the similarity between traces and ascertains if TF is more similar to VF or PF. Globally, the results pinpoint a high accuracy rate of 93.31%.

Method Based on Ratcliff/Obershelp

The authors highlighted a method based on a string similarity algorithm to map software product names from system logs to product names in VDBs [5]. The proposed techniques involve gathering software product names from several target systems using the Winapps Python library. Then, these software names are mapped to the CPE entries of VDBs using the Ratcliff/Obershelp algorithm. Next, potential vulnerabilities are found in the NVD database based on the associated CPE. Finally, CVSS scores are attributed to the detected vulnerabilities based on published metadata. We note that the proposed technique and tool demonstrated an average accuracy of 79%.

Method Based on CTPH

A new study proposed a method for VULnerability DEtection method based on Function Fingerprints and code differences (VULDEFF) to detect vulnerabilities in software source code by detecting the differences between patched and unpatched software [50]. VULDEFF consists of three modules: (i) data preprocessing, responsible for the collection and processing of vulnerability patches and source code into a dataset. Then, (ii) generating function fingerprints using the Context Triggered Piecewise Hashing (CTPH) algorithm and CRC32 checksums [51]. In the last module, (iii) VULDEFF integrates fuzzy matching (size, character repeat, longest common substring, weighted edit distance and scale edit distance) to compare the syntactic structure of vulnerable function (VF) and target function (TF) to identify potentially vulnerable cloned code.

Method Based on Jaro–Winkler

This method revealed that there is a lack of published CPEs for every vulnerability library integrated into VDBs, such as NVD [52]. The majority of the affected products, according to the author, are released in an unstructured manner, which complicates automated analysis. To overcome the previous issues, the study proposes to automate the construction of CPEs for vulnerable products listed in non-NVD security advisories. To this end, the main focus consists of performing a string-matching similarity between unstructured vendor names and structured vendor names in the CPE dictionary. The author evaluates five string similarity metrics (Levenshtein, Discounted Levenshtein, Jaro, Jaro–Winkler, and Ratcliff/Obershelp). Next, this study suggests an alternative Jaro–Winkler algorithm that modifies the weight of each token in the advisory’s vendor name based on its frequency in a specialized corpus. By building accurate CPEs of software libraries, the process of detecting vulnerabilities will depend on published security events in VDBs. Although the result is promising, some limits are still raised.

Method Based on GPT

A recent study presented a reflection on the ability of four GPT languages, such as GPT-3, GPT-3.5/ChatGPT, GPT-4, and Bing Chat-Bot, to accurately answer key VMS-related questions, such as the CVSS of detected vulnerabilities, their vectors, how to mitigate them, the affected products, and information about mitigation and exploit [53]. To this end, the author performed an empirical study on retrieving CVSS scores and vector and CPE information by using GPT models. The result is shown to be incomplete and inaccurate and is revealed to be different from NVD data. The study also found significant limitations in the models' ability to gather information about mitigation and exploits, especially for complex data. However, LLMs showed high accuracy and low hallucination rates when summarizing vulnerability information from the full text of advisories.

Method Based on HermeScan

The current method discussed an approach to detect taint-style vulnerabilities (security issues in data flow) in Linux-based IoT firmware by applying the reaching definition analysis (RDA) technique [54]. HermeScan starts with extracting firmware and libraries to identify user input and system operations, as shown in Figure 15 below. Then, it builds a comprehensive Control Flow Graph (CFG) to precise untrusted data enters and sink points (critical operations occur). Next, the method employs fuzzy matching technique between the front-end and back-end to uncover untested candidate functions. By using Reaching Definition Analysis (RDA), the core of HermeScan analyzes and tracks data flows between functions to handle control complexity. Finally, the taint inspection engine verifies any security policy violations.

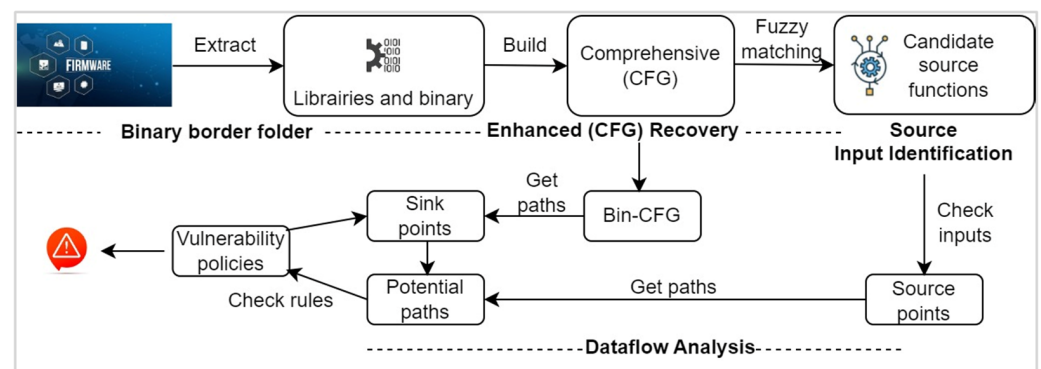


Figure 15. Overview of HermeScan. (Adapted from [54]).

Table 3 below provides additional information about the different methods used in this category.

4.1.2. Finding Analysis

The method based on RE identifies vulnerabilities and required patch releases without providing specific details on the accuracy rate. Challenges related to CPE and CVE metadata impact the results. To address these issues, the Levenshtein technique uses a semi-manual approach in the CPE matching process, achieving approximately 83% accuracy, as 10 out of 12 products were correctly matched. Manual intervention was necessary due to the error-prone nature of fully automated approaches. However, inconsistencies and incomplete data published in CVEs still affect the overall accuracy of the studied methods.

Building new CPEs from banner texts achieved a high accuracy of 98.9%. Despite this success, issues such as overmatching, short product names, or common names led to false positives or missed vulnerabilities. In the CVE matching process, the TF-IDF-based keyword extraction pipeline was used to identify the most affected software, with 70% of vulnerabilities (around 57,640 CVEs) accurately identifying full software names.

Table 3. Collected methods related to the similarity matching-based approach.

Authors, Year	Comparison Method	Scope or Ecosystem	Limitations and Challenges	Human Interaction (HI)	Attributes	Prioritization	Scanning Mode
Gawron et al., 2017 [46]	Regular Expression	IT	Incomplete information in log file; No matching between CPE ID/products and CPE/VDBs; Vulnerability without CPE; Vulnerability zero-day.	No	CPE, log file, HPI-VDB, OSVDB, NVD.	No	Passive
Sanguinoc and Uetz, 2017 [9]	Levenshtein edit distance	IT	Mismatch errors; Similar semantic CPE with different syntax; Large and complex computation; Human intervention is labor intense; CVE description without software product metadata.	Yes	Vendor, Product and version. CPE, CVE.	Yes	Passive and active
Na et al., 2018 [47]	Building CPE for connected devices	IoT	Dependence on banner text quality, and complexity in managing vague or incomplete data. Deprecation in CPE dictionary; No CPE entries in the CPE dictionary.	No	Banner text, CPE (Product and vendor name).	No	Passive and active
Elbaz, Rilling, and Morin, 2020 [48]	TF-IDF	IT	Heavily dependent on the quality of text description and in case of lack of relevant keywords, the results may lead to false positives or negatives. Analysis based on description only may output errors; Incomplete metadata in VDBs represent a considerable issue; Limited heuristics may cause occasional inaccuracies.	No	Free-form description, keywords extracted from CPE URI, CPE, CVE.	Yes, the result is the most probable affected software.	Passive

Table 3. Cont.

Authors, Year	Comparison Method	Scope or Ecosystem	Limitations and Challenges	Human Interaction (HI)	Attributes	Prioritization	Scanning Mode
Xu et al., 2020 [49]	Basic bloc mapping, Greedy Algorithm, Levenshtein distance Algorithm.	IT and software used in IoT devices	BinXray relies on the accurate function matching, as well as a dependance on a binary compiled system; A challenge is raised when a function receives multiple changes at the same location in different versions; Complex and large functions may increase the time consumption for analysis; Remain noise to impact the accuracy.	No, but manual analysis is required to analyze potential vulnerable functions and then, check ambiguous cases.	Vulnerable function (VF); Patched function of a program (PF) and target binary program.	No	Passive
Ushakov et al., 2021 [5]	Ratcliff/Obershelp	IT	Name inconsistency issues during the collection of software products; Error-prone mapping due to the obtained score; Manual verification is required in certain steps; Common issues related to the VDBs.	Yes, in some cases.	Vendor, Product and version, CPE, CVE.	No	Passive and active
Zhao et al., 2023 [50]	Fuzzy matching; Hash algorithms (CTPH and CRC32); Weighted edit distance and Cuckoo filter, and AST.	IT	Extracting and analyzing Abstract Syntax Trees (ASTs) may increase the computational cost in a complex infrastructure; Patching methods differ and could generate false positives; VULDEFF focuses only on syntactic and structural features without handling semantic aspects; The balance between the three thresholds (ξ_1 , ξ_2 , and ξ_3) should be well set to avoid impacting the accuracy of VULDEFF.	No, but in case of false positive or ambiguous results, validation is required to maintain the accuracy of VULDEFF.	Target function (TF), patch function (PF) and vulnerable function (VF), dataset of vulnerable function and patches.	No	Passive

Table 3. Cont.

Authors, Year	Comparison Method	Scope or Ecosystem	Limitations and Challenges	Human Interaction (HI)	Attributes	Prioritization	Scanning Mode
McClanahan et al., 2023 [52]	Jaro–Winkler; NLTK snowball stemmer; Cleanco Python library;	OT	The variability in vendor names impacts the accuracy of the matching process; Vulnerabilities published without software description or no CPE at all; Handling abbreviations and acronyms when building exact CPEs; Handling Jaro–Winkler errors during the matching process; Following versioning names over time; Labor-intensive in building the dataset.	Yes, especially for building the dataset.	Dataset of ICS advisories published before July 25, 2023; CPE, CVE.	No	Passive
McClanahan et al., 2024 [53]	GPT-3; GPT-3.5; GPT-4; LLM and Bing chatbot	Linux system	GPT-3 and GPT-3.5 are not accurate in finding CVSS scores, vectors, and affected products; GPT-4 and Bing chatbot still had issues retrieving correct and precise CVEs; LLM is prone to hallucinations.	Yes, to interact with user-prompted questions.	CVE, CPE, CVSS, Exploits, Mitigation, Google, and NVD.	No	Passive
Gao et al., 2024 [54]	Fuzzy matching, CFG; RDA	IoT	Build incomplete CFG for complex firmware (obfuscated code or indirect calls); Many interdependencies between functions and libraries may require more computations and resources; Dynamic; Over-tainting constitutes a challenge and leads to incorrect vulnerability reports	Yes	IoT device firmware; Shared libraries; Binary files; 0-day dataset; N-day dataset.	No	Passive

The Ratcliff/Obershelp algorithm contributed similarly to the regular expression-based method, matching system logs with CPE/NVD data to identify affected software, achieving an average accuracy of 79%. Additionally, several solutions focused on resolving incomplete CPE listings. The modified Jaro–Winkler technique achieved 83.7% accuracy in vendor matching surpassing the Levenshtein edit distance and Ratcliff/Obershelp methods.

Recently, the method based on ChatGPT was evaluated for retrieving CVSS scores, identifying affected CPEs, and offering mitigation strategies. However, these techniques struggled with providing accurate results. Lastly, the HermeScan technique, which integrates fuzzy matching strategy to detect vulnerabilities caused by insecure data flows in firmware, achieved a true positive rate (TPR) of 81%. These findings suggest that both the modified Jaro–Winkler and HermeScan methods present promising results in the field. It is worth noting that the quality of published security event data impacts the accuracy of vulnerability detection methods used in this process.

Methods based on matching techniques rely on the performance of the selected algorithm. Various algorithm matching is employed in the previous methods, such as Jaro–Winkler, fuzzy matching, Ratcliff/Obershelp, and Levenshtein edit distance, among others. In the practical case, a new version of Jaro–Winkler was used to match vendor names between the CPE dictionary and security advisories. As a result, it can match “renewable energy laboratory (nrel)” to “nrel” provided by CPE vendors, while it faces issues in finding compatibility between “schweitzer engineering laboratories” and “selinc”.

Thus, all the previous matching methods can be used separately or together to detect potential vulnerabilities in IT assets. Next, we will introduce specific methods used by the graph approach.

4.2. Graph-Based Approach

Based on graph theory, DBs, and other AI techniques, the second basket is another method for modeling and analyzing the relationships and interactions between elements inside a target system. This approach plays a significant role in vulnerability detection by utilizing complex relationship mapping. It includes multiple inputs, fast traversal of linked security data stored, scalability, real-time analysis of security events, and simulation and prediction of potential cyberattacks, as depicted in Figure 16. More details about these methods are presented throughout the synthesis of several methods existing in the literature review.

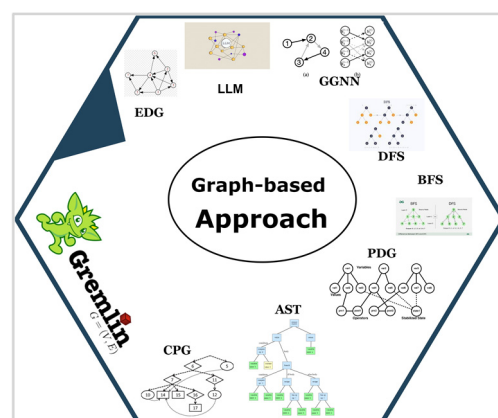


Figure 16. Features of graph-based approach.

4.2.1. Graph-Based Approach Methods Description

Method Based on GGNN

This study introduced in their work a novel framework named FUNDED (Flow-sensitive vUInerability coDE Detection) [55]. It combines a graph-based learning concept with automated data collection for code vulnerability detection. FUNDED leverages the

benefits of advanced graph neural networks (GNNs) [56] to represent the target program as a graph by AST [57], capturing control, data, and call dependencies, using PCDG [58], to enhance code vulnerability detection. The framework first converts the program source code into a graph representation, where nodes represent statements and edges represent various code dependencies (control, data, and call). In the first phases, FUNDED includes Gated Graph Neural Networks (GGNNs) [59] to capture complex code structures and relationships critical for identifying vulnerabilities. In the second step, related to data collection, the framework gathers high-quality training samples from open-source projects to identify vulnerable code and enrich the training dataset with real-life examples. A key aspect of this phase is the combination of expert models (support vector machine (SVM), random forests (RFs), k-nearest neighbor (KNN), logistic regression (LR), and gradient boosting (GB) to identify vulnerability-relevant commits. Conformal Prediction (CP) measures the statistical confidence of each expert model's predictions. The third step highlights the multi-relational graph modeling technique, which helps create multiple relation graphs for different types of edges (e.g., control flow, data flow, syntax). It aggregates information across these relation graphs using a Gated Recurrent Unit (GRU) [60] to learn a comprehensive representation of the program. Finally, the last step involves training the models on real-life samples and applying transfer learning to adapt the model to different programming languages, as shown in Figure 17. Through the utilization of the trained model and the learned graph representations, the proposed solution assists in identifying patterns indicative of vulnerabilities.

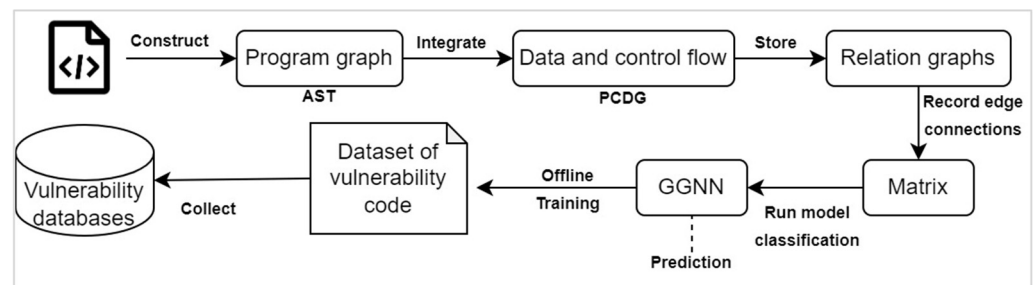


Figure 17. Workflow of FUNDED. (Adapted from [55]).

Method Based on SPG

Using static program analysis approaches, this contribution presented a method for handling software vulnerability detection based on the Slice Property Graph (VulSPG) [61]. After parsing the target code using the open-source tool joern (<https://joern.readthedocs.io/en/latest/>, accessed on 31 July 2024), the method matches some vulnerability candidate syntax characteristics by applying the Abstract Syntax Tree (AST) [57], following six types of Syntax-based Vulnerability Candidates (SyVCs) [62]. Then, the Program Dependency Graph (PDG) [63] is traversed to obtain slice nodes. Additionally, the Code Property Graphs (CPG) [64] generate data and control-dependencies as well as function calls among slice nodes to build the Slice Property Graph (SPG). The second step encodes the semantics in the SPG nodes. This process involves lexical analysis via the Word2Vec model and semantic feature vectors through a token-level attention mechanism [65]. This type of information is then combined to enrich the node's feature representation and output an embedded vector of the graph nodes. The current step divides the SPG into three types of subgraphs: a Control Dependency Graph (CDG), a Data Dependency Graph (DPG), and a Function Call Dependency Graph. The Graph Encoding Network phase then uses Relational Graph Convolutional Networks (R-GCNs) [66]. This helps to learn the hidden state of each node in each layer and concatenate them to capture the comprehensive graph features essential for identifying potential vulnerabilities. In the last step, the method proposes a subgraph-level attention mechanism to obtain the feature vector of the combined subgraphs (vectors).

Finally, the obtained subgraph and SPG are concatenated into a classifier network for vulnerability detection.

Method Based on Methods and Gremlin Graph

In the same year, the current study proposed a method based on three broad blocs [67]. The first bloc is based on the graph model to store CVE information (JSON format from NVD feeds) and the related applicability statement Conjunctive Normal Form (CNF), along with a hierarchy of asset configurations [31,36]. Then, there is an insertion procedure step that pre-processes CVE and CNF (as vertices: CveVertex into the graph) while performing attribute-value pair comparison. Finally, the graph search query block finds all vulnerable pairs of CVEs, using a Gremlin-based graph, in a single traversal.

Method Based on EDG

This method highlighted a new technique based on the Extended Dependency Graph (EDG) model for vulnerability analysis in space (asset) and time (recurrence) within the Industrial Automation and Control System (IACS) environment [68]. This method involves various crucial processes to alleviate cyber risk, as shown in Figure 18. The first process constructs a direct graph to represent, map, and understand the dependence between assets, CVE and CVSS, CWE, and attack pattern (CAPEC) [69]. By integrating quantitative metrics, the proposed approach prioritizes updating and upgrading activities and ensures continuous monitoring of the configuration system. Vulnerability management identifies the global CWE in the target system and brings out insights related to the root causes of the detected vulnerabilities. The EDG model dynamically adapts by covering the entire device lifespan within a complex environment and under System Under Test (SUT) as an industrial component, following the denomination in the ISA/IEC 62443 standard [70]. By the end, the method provides visual outputs representing the system's security posture and produces detailed reports. Despite positive results from evaluating and experimenting with the OpenPLC project (open-source Programmable Logic Controller (PLC), both in software and hardware) [71–74], the approach could be strengthened by integrating a mathematical model to combine each asset's CVSS metric values, enhancing patch prioritization, and using other techniques to predict future vulnerabilities.

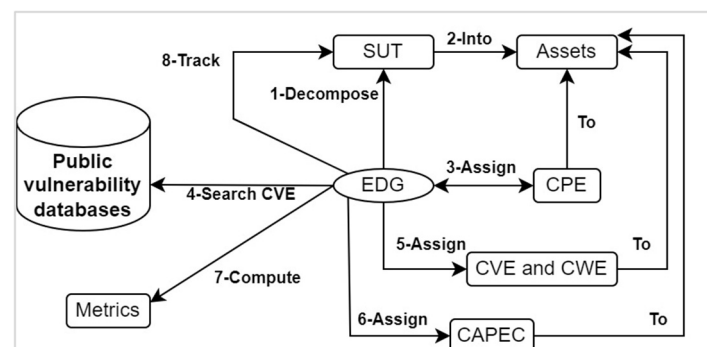


Figure 18. Steps to build EDG for SUT. (Adapted from [68]).

Method Based on Analytic Graph

This contribution introduced a Graph-Based Analytic method to improve Cyber Situational Awareness (CSA) across complex computer networks [75,76]. The CSA operates on three levels: perception, comprehension, and projection of situations in a cyber environment. The authors introduce graph-based intelligence, which leverages the second level of CSA. This method starts by identifying hosts near compromised devices using Depth First Search (DFS). Next, this method discovers vulnerable assets using breadth-first searching (BFS) to identify and manage network vulnerabilities. Likewise, community detection and frequent subgraph mining (FSM) algorithms segment the network as part of the proactive

security measures in Incident Response IR [77]. Ultimately, graph centrality measures (Degree—Betweenness—PageRank—Closeness) prioritize nodes based on their influence, impact, critical subnet, and other relevant parameters to assess network security [78,79].

Method Based on Threat Knowledge Graph

In this contribution, the method is proposed to aggregate extracted security information like CVE, CPE, and CWE to predict the associations between these elements [80]. When gathering data from external sources, the approach proceeds to graph construction, where nodes represent the entities (products, vulnerabilities, and weaknesses) and edges represent the relationships between them. Then, the knowledge graph is optimized through data pre-processing and enhancement before applying the TransE model to predict new or missing associations between entities [81]. Moreover, this approach evaluates the prediction using rank-based metrics such as Mean Rank (MR), Mean Reciprocal Rank (MRR), and Hits@N scores to ascertain the accuracy of the result [82,83]. Finally, the method is tested on closed-world settings (only known associations) and open-world settings (predicting associations). The results are promising, but further investigation is needed to improve performance.

Method Based on LLM

A recent study presented the method GRACE, which empowers graph structural information with LLM to learn models on data dependencies and incorporate specific domain knowledge to enhance LLMs' performances for software vulnerability detection [84]. The semantic, lexical, and syntactic similarity aspects of the most similar code are considered to provide better demonstrations for in-context learning. GRACE's first module, the demonstration selection module, begins with a semantic comparison between source code and input code, then considers lexical and syntactic similarities to generate the most similar code. As a second module, graph structure representation utilizes Control Flow Graph (CFG: possible traversed path), Abstract Syntax Tree (AST: Syntactic form) and Program Dependence Graph (PDG: data and control dependencies) to understand structures, complex relationships, and dependencies within the input code [57,58,85]. Ultimately, the last module highlights the vulnerability detection module, which consists of two components: (1) The basic prompt outputs result from binary classification (vulnerable or not), and the outcomes are more accurate when adding domain information in this context. (2) The auxiliary information reflects the in-context learning demonstrations (the results of the first module enhance the vulnerability detection capabilities of LLM) and graph structure information (a more comprehensive understanding of the code's structure). Globally, the results are promising; however, further evaluation in different environments is needed to assess their effectiveness and be more accurate.

Method Based on Attack Graphs

This study proposed an approach to analyze and assess IoT networks' security posture [86]. IoT systems face security challenges in dynamic environments, where frequent network topology updates (newly added devices and multiple interconnections) are highly manifested. The practical case in this study highlighted body sensor networks, which constantly changed their location, exposing them to various forms of cyberattacks. To address this issue, this study combines graph database usage (Neo4j) [87], reachability (directed path between nodes), and attack graphs (possible attack paths) to provide a real-time model for analyzing CVEs and attack propagation. The graph node represents devices and associated CVEs, while the edge captures the relationship between them. Attack and topology graphs are generated by using Cypher queries. This method consists of generating an updated network's topology when new devices join or leave. Then, the reachability is automatically calculated, and finally, the attack graph is given considering new device CVEs and the network's current configuration. As a result, the model contributes to assessing potential risks and provides real-time insights related to possible attack paths and CVEs. More details are shown in Table 4.

Table 4. Collected methods related to graph-based approach.

Authors, Year	Used Method	Scope or Eco-System	Limitations and Challenges	Attributes	Human Interaction (HI)	Prioritization	Scanning Mode
Wang et al., 2020 [55]	GGNN: (PCDG, AST, GRU) Mixture of Expert Model (SVM, RF-KNN, LR and GB-RE).	IT	Data: Many commits in open-source projects includes benign code snippets in the training samples; Data quality assessment: The check process remains manual; ML models: Are dependent on the quality of dataset which needs continuous upgrade; Uncertain situations: the models are predefined to produce high-probability answers which may lead to false positives; Resource-intensive: More time to perform training a huge volume of data and learning from these graphs.	Target: Program Source Code; Data: CVE, NVD, SARD and open-source projects hosted on GitHub; Dataset: SAP [88] for Java and ZVD [89] for C/C++; Expert Models; Conformal Prediction (CP).	Yes, especially in data gathering process, Initial sample labeling (inspecting and labeling) and in continuous learning where predictions are reviewed by developers to provide ground-truth labels.	No, a Binary Decision is given by function detection.	Passive
Zheng et al., 2021 [61]	SPG: R-GCN-AST-PDG-CPG.	IT	SARD [90] and NVD [36] datasets: present noise and irrelevant information, inconsistencies in training data, inaccurate synthetic samples, limited coverage of vulnerability types for training; SPG: complex to construct, semantic process is resource; Intensive, reducing redundancy can lead to omission of potentially relevant information; Handling variability in code structure impacts effectiveness of SPG generation; VulSPG is focused only on vulnerability detection in programs written in C/C++.	Source code; Outputs of PDG: Data and flows of the program; Semantic Outputs of CPG by using AST and CFG; Syntactic features: slicing criteria to generate (SPG).	Yes, to handle complex interpretation of results, to perform a validation of vulnerability source code, to adjust the model parameters and refine the slicing criteria as well as alter dependencies in SPG construction process.	Yes	Passive

Table 4. Cont.

Authors, Year	Used Method	Scope or Eco-System	Limitations and Challenges	Attributes	Human Interaction (HI)	Prioritization	Scanning Mode
Tovarnak et al., 2021 [67]	Graph-based methods and Gremlin graph traversal language	IT	Granular details of asset configurations increase the complexity of assets management; Frequent alteration in configurations system and in VDBs; Intensive computation when applying to a large-scale ecosystem; Complete dependence of the accuracy of CVE and CPE published.	Known CVE vulnerabilities (Json Format), and CPE applicability statements (Version 2.3 reference implementation [30,31]).	Yes, for vulnerabilities and device fingerprints, but HI is required again in updating CVE or asset data or modifying the graph structure.	No	Passive
Longueira-Romero et al., 2022 [68]	EDG model (directed graphs and dynamic tracking); Quantitative Metrics (CVSS-based Metrics and Continuous Assessment)	OT (IACS)	Global dependence of input data accuracy (CVE and CWE); Complexity in managing dynamic updates or upgrade (CVE, patch or firmware); Resource intensiveness: in maintaining EDG model; The used model loses effectiveness in front of the unknown or (zero-days) vulnerabilities.	All CPE under the SUT; Public CVE, CWE and CAPEC; Time-quantitative metrics based on CVSS: for vulnerabilities (M0 to M6) and for weaknesses (M7 and M8).	All the process included in this approach are automatic; nevertheless, periodic reviews may require manual input to ensure accuracy and relevance.	Yes, especially for patching activities.	Passive

Table 4. Cont.

Authors, Year	Used Method	Scope or Eco-System	Limitations and Challenges	Attributes	Human Interaction (HI)	Prioritization	Scanning Mode
Husak et al., 2023 [75]	Graph-based analytic-graph traversal (DFS and BFS), Community detection, FSM, and graph centrality measures.	IT (Network)	New paradigms (new query languages and adaption to data processing); Lack of comprehensive datasets (high-quality datasets for training and validating graph-based cybersecurity systems); Need for unified ontology (The effectiveness can be limited); Explainability and complexity (difficulties for users to understand and interpret the results).	Network hosts, users, services information, IP addresses, vulnerabilities of CVE (CPE included), and security events; Nmap for scanning (CPE string) and the Neo4j Graph Data Platform for storing and visualizing the data [87].	Yes, for data interpretation, incident response, decision making and maintenance and updates.	Yes	Passive and Active
Shi et al., 2023 [80]	Threat knowledge graph (Translating Embeddings: ML model TransE)	IT	Dependence of the external cyber security event; Incomplete vulnerability information or delayed updates; Managing prediction errors and maintaining complexity; Manual analysis is required; The prediction of the association between entities is based on historical data; other newly entities may represent an issue.	CVE, CPE, and CWE from NVD.	No, but in the set-up and defining parameters of the model, human expertise is required to interpret the result.	Yes	Passive

Table 4. Cont.

Authors, Year	Used Method	Scope or Eco-System	Limitations and Challenges	Attributes	Human Interaction (HI)	Prioritization	Scanning Mode
Lu et al., 2024 [84]	Graph Structural Information Integration (AST-PDG and CFG); LLM (in-context learning); CodeT5 [91] to extract semantic features; T-SNE [92] to reduce feature dimensionality; -SimSBT [93] to generate sequences during the traversal path.	IT	Higher computational costs and resource demands for building a complex graph representation in high-scale ecosystem; Dependence on quality during the in-context learning and domain-specific information; Effectiveness GRACE with other programming language; Certain nuanced or complex semantic information may impact the detection of some vulnerabilities; New vulnerable patterns not existing in the data source.	Tree datasets are used to train models in detection if the code is vulnerable or not. FFmpeg [94] and Qemu [95] and Big-Vul [96].	Yes, the three modules integrated are fully automated.	No	Passive
Salayma 2024 [86]	Neo4j, Cypher queries.	IoT	Issues within a large and complex IoT environment; The reachability and attack path computations can face limitations when firewall policies grow in complexity; Dependence on Neo4j and its cypher query language may limit the portability of the solution to other graph databases.	CVE, Attack paths.	Yes, to elaborate queries.	No	Active

4.2.2. Findings Analysis

In this second approach, the Funded method combines a graph-based learning set and automated data collection to detect vulnerabilities in code sources. This study achieved a high accuracy (92%) in function-level vulnerability detection, which surpasses matching-based methods. Building on this principle of Funded, VulSPG incorporates rich semantics and explicit structural information to enhance the performance of vulnerability detection in target code functions, achieving a slightly improved detection accuracy of 93.8%. Next, the method based on the Gremlin framework manages asset component trees and their related vulnerabilities in assessing the vulnerability process. Although no specific accuracy metrics are provided for this method, it offers a qualitative improvement by efficiently mapping vulnerabilities to asset components. Based on graph structure representation, the analytic graph focused on real-time monitoring, defense preparation, and response to incidents, although concrete performance metrics like accuracy are not explicitly reported. The threat knowledge graphs method, which reveals hidden relationships within CPE, CVE, and CWE, show promise with a good Mean Reciprocal Rank (MRR) score of 0.424, indicating their effectiveness in uncovering vulnerabilities. Recently, the GRACE method combined graph structure and LLM to improve software vulnerability detection. The method resulted in a significant improvement in F1 scores, 14.82%, 24.64%, and 73.8% across three different datasets, showcasing its effectiveness in various contexts. Lastly, the attack graphs method integrates network topology and reachability graphs to detect new attack paths, especially in dynamic IoT environments. This technique shows efficiency in dynamically adjusting IoT and significantly improves the detection of new attack paths. Based on prior findings, the GRACE method demonstrates significant potential in this area.

In practical cases, the method based on the threat knowledge graph uses knowledge graph embedding (TransE) to predict more associations between CVE, CPE, and CWE. An amount of 465 affected products were analyzed by this method, which revealed only 11 false positive predictions. More specifically, using a threat knowledge graph to analyze the CVE-2021-21348 before 4 August 2021, which affects the Java library XStream, the model predicted various other products, such as Debian Linux and Oracle, among others, which were only revealed after the cutoff date.

Thus, the graph-based approach combines several graph methods to build a global dependance and relationship between concerning elements to detect vulnerabilities and improve cybersecurity awareness. The following subsections provide more details on the provided features.

4.3. Feature Modeling-Based Approach

This category of approaches focuses on feature modeling concepts that support the representation of components within a Software Product Line (SPL) [97]. The proposed technique in the context of cybersecurity addresses security variability to identify potential vulnerabilities and detect cyber risks. Feature Modeling (FM) handles system configuration to represent pertinent attributes of each element, capture relationships between those attributes, synthesize all dependencies and constraints, discover potential variabilities among software systems, and then reason about the compactly represented systems' possible configurations [98]. This approach ensures comprehensive coverage of the target system and the dependency management tasks required, as shown in Figure 19. This helps to identify indirect vulnerabilities by assessing how suspicious changes in one system element may impact other components. In order to improve cyber detection accuracy, the FM makes use of its knowledge of syntactic and semantic properties of the code to identify hidden and context-specific vulnerabilities. Next subsections provide more details on the previously provided features.

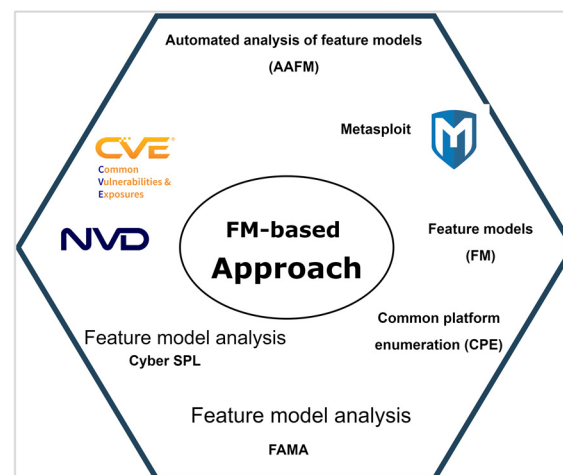


Figure 19. Features of FM-based approach.

4.3.1. Feature Modeling-Based Approach Methods Description Method Based on CyberSPL

A new contribution presented a Cyber Software Product Line (CyberSPL) solution [99]. It offers a way to assess cybersecurity policies based on possible configurations. To represent the configuration parameters of software systems, the modeling configurations are based on feature models tailored to specific cybersecurity domains [100]. These models outline a variety of requirements, relationships, and dependencies that must be adhered to ensure cybersecurity compliance. Next, CyberSPL uses Constraint Satisfaction Problems (CSP) to transform feature models into formal representations, verifying the model's satisfaction and figuring out the number of legitimate software products [101]. Following the verification process, CyberSPL uses analysis activities and reasoning techniques, notably ChocoSolver, to reason with feature models [102]. This approach diagnoses the system setup and identifies any non-compliant configurations. Therefore, this outcome continues to be an anticipatory cybersecurity measure to identify and fix vulnerabilities before exploitation by cyberattacks. For this purpose, the CyberSPL is designed to be updated against the latest cybersecurity policies. The Figure 20 below illustrates the global workflow of CyberSPL and outlines major outputs. Finally, CyberSPL, which is connected to the FAMA framework via an REST API, was evaluated to handle Apache Server Configuration, Linux Kernel Security, Android Security Settings, and SSL/TLS protocol settings [103].

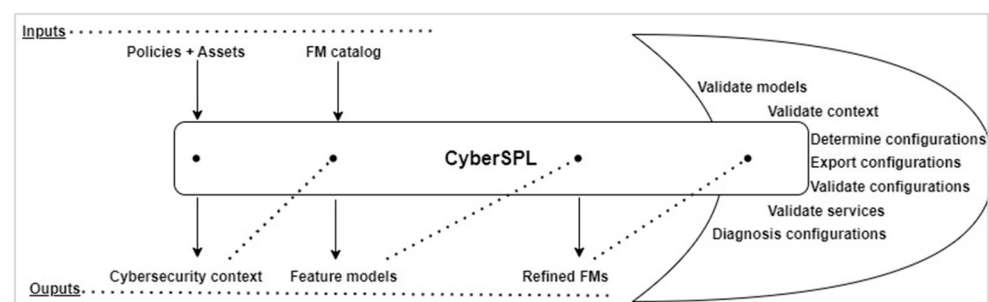


Figure 20. CyberSPL workflow. (Adapted from [99].)

Method Based on Attack Scenario

Another work presented a study focusing on the integration of feature modeling to support security assessments by virtualizing attack scenarios for software systems [104]. In this context, the methodology starts with extracting security events from VDBs, linking them, and correlating dependencies between software systems. Next, a feature model is built to capture vulnerabilities and the relationships between them. At this step, actions are

carried out manually by pulling from the Metasploit Framework (MSF) and vulnerability databases to build records for each attack scenario into a vulnerability feature model [105]. For evaluating the presented work, the authors built the relationship between Firefox and operating systems using leaf features and 24 cross-tree constraints. The next step involves integrating the retrieved data to replicate vulnerable systems. The virtualized systems are then attacked using MSF scripts before evaluating the scenarios' effectiveness. This capability allows all security stakeholders the opportunity to identify pertinent attack scenarios and vulnerabilities for their purposes.

Method Based on AMADEUS

A recent solution unveiled a new FM approach called AMADEUS (AutoMAteD sEcU- rity teSting) [106]. This solution automates the examination and testing of cybersecurity vulnerabilities in feature-model-based configuration systems. The initial contribution is integrating vulnerability management with FMs. At this level, AMADEUS operates with two modes during the reconnaissance and enumeration phases: the custom mode allows users to provide a list of relevant keywords about a target system, while the input list is automatically extracted by the Nmap tool. Then, using the previous keyword, AMADEUS integrates a web scraper module to extract the CVE ID from NVD. These results are used to gather all possible vulnerable configurations (CPE) for each CVE ID. Following that, three methods are used to generate specific FMs: sub-FM/vendor, sub-FM/running configurations, and sub-FM/single FM tree. These algorithms link the configuration of the target system to the security events that are extracted from vulnerable repositories. The FM building process retrieves unrestricted FM from CPEs by using FaMa and incorporating cross-tree relations to adjust FM variability according to the restriction of CPE attributes and the running configurations [103]. The final stage focuses on the reasoning concept of the built FMs, which includes (i) generating attack vectors based on vulnerable configurations conjugated with a set of all products of the model; (ii) determining if a specific configuration is vulnerable; and (iii) prioritizing attack vectors according to a specific criterion.

Method Based on AMADEUS-Exploit

Continuing the research in the same area, a similar work enhanced the previous work by adding an exploit layer to the AMADEUS framework and incorporating additional vulnerability repositories to consider exploits and improve vulnerability management [107]. Then, the AMADEUS core uses a new engine to improve vulnerability analysis and FM reasoning capabilities. During the AMADEUS-Exploit investigation study, a real-world scenario was adopted to evaluate the method's capabilities. This new methodology, integrated with FaMaPy, reflects the ability to display variability concerning CVEs, CPEs, and exploits to enhance the unified global monitoring system and improve automatic analysis mechanisms [108]. The workflow in this study involves three stages: (i) discover target elements, which involves using active and passive tools to manually and automatically build a system inventory; (ii) vulnerabilities and exploits identification searches for CVE ID, CPEs, and exploit ID from NVD, VulDb, and ExploitDB; (iii) assess vulnerabilities and exploits, which generates a catalogue of valid FM, considering vulnerabilities, configurations, and exploits, as shown in Figure 21. This process correlates the dependencies between vulnerabilities and exploits to develop multiple reasoning operations. This technique helps set vulnerability management priorities.

Further details about the previous methods are summarized in the Table 5 below.

Table 5. Collected methods related to feature models (FM).

Authors, Year	Used Method	Scope or Eco-System	Limitations and Challenges	Attributes	Human Interaction (HI)	Prioritization	Scanning Mode
Varela-Vaca et al., 2019 [99]	FAMA framework-REST API; ChocoSolver -CSP.	IT	High initial effort: assets cartography and security control identification; Dependency on accurate models: Any error may lead to incorrect diagnosis; Manual updates of FM are required.	Cybersecurity policy, Assets- Cybersecurity Context.	Yes	Yes	Passive
Kenner et al., 2020 [104]	In this study, throughout the attack scenarios and penetration testing stage, only the specific MSF is defined.	IT	Security events: Lacks quality, difficulties in extracting relevant data and inconsistencies issues; Analysis, extraction, synthesis, and date are performed manually; Additional manual analysis is required to build FM; During the evaluation, errors or technological issues relating to constraints on the environment occur; The suggested model must be heavily modified for many use cases with the goal to be reusable; Maintainability and real-time updates require additional effort to be accomplished in the event that a software system changes.	Vulnerability Databases: NVD. Exploit Databases. Attack Scenario Dataset and Framework: MSF.	Yes	Yes	Passive

Table 5. Cont.

Authors, Year	Used Method	Scope or Eco-System	Limitations and Challenges	Attributes	Human Interaction (HI)	Prioritization	Scanning Mode
Varela-Vaca et al., 2020 [106]	FaMa; FM: fm.py; Tool: Nmap; web scrapers: scraper.py.	IT	Dependence: relevant key work addition requires to be manually included to enhance accuracy; Assets inventory depends only on NMAP scan results which may contain inconsistencies or omission; Difficulty to manage products whose CPE does not meet specifications and that NMAP is unable to identify; VDBs: inconsistencies and relevant data omission can affect the accuracy of the FMs; There are more cross-time limitations when a significant number of features (CVE and CPE) are included; The FM does not accurately represent the state of assets in terms of RC and CPE; System feature detection is still manual; It will be time-consuming as a result of the scraping mode carried out in a large complex environment.	Vulnerability Databases: NVD; CPE; Running Configuration RC (environments in which the vulnerability can be reproduced); Reports from infrastructure analysis (ports, services, etc., ...).	Yes	Yes	Passive and active
Varela-Vaca et al., 2023 [107]	FaMaPy; Tool: Nmap; web scrapers: scraper.py and exploitdb; scraper.py; FM: fm.py;	IT	The AMADEUS-exploit still has the same limitations as the AMADEUS framework; Exploit DB: Incomplete, inconsistent, or error data may affect the accuracy of FMs; Misinterpreting the automated analysis and FMs' reasoning; Need more external validation experts.	NVD, ExploitDB and VulDB; CPE, RC, and key terms.	Yes	Yes	Passive and active

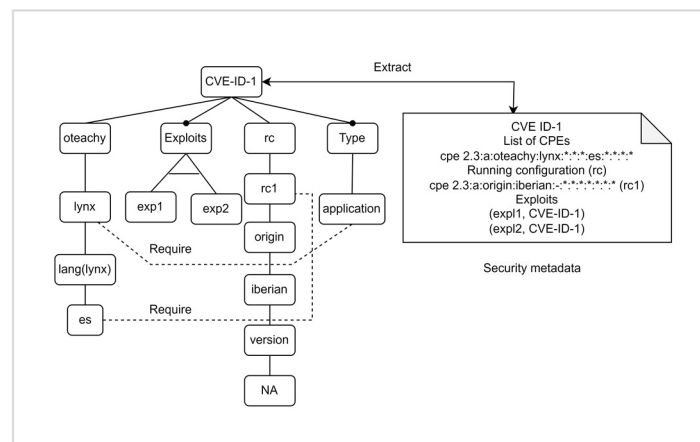


Figure 21. Example of FM construction used by AMADEUS and AMADEUS-Exploit. (Adapted from [106,107]).

4.3.2. Findings Analysis

CyberSPL assists cyber professionals by automating the analysis of non-conformance with cybersecurity policies. This method combines feature models’ capacity with automated verification and diagnosis. The evaluation showcased performances in operational development (DevOps). Additionally, the method based on attack scenario incorporates feature model variability to represent the vulnerability of the target system. It integrates attack scenarios to uncover insights about potential exploitation areas. The evaluation showed that 5 out of 18 attacks failed to exploit the identified vulnerabilities, with detailed reasons listed in Table 5. In addition, the AMADEUS method integrates the SPL techniques, used in CyberSPL, with feature models to automate the infrastructure inventory analysis, scraping vulnerability databases, extracting vulnerability configuration, and inferring possible attack vectors. Despite some limitations, the evaluation demonstrated high accuracy in generating and validating attack vectors. In the same field, AMADEUS-Exploit, an extension of AMADEUS, adds an exploit layer to feature models and improves reasoning capacities. This method was evaluated in a real scenario, identifying 4000 vulnerabilities and 700 exploits. Generally, AMADEUS-Exploit has proven more its scalability and efficiency in vulnerability detection and management.

In practical case studies, the previous methods can be employed within enterprises to identify, assess, and prioritize vulnerabilities in this infrastructure. For AMADEUS-Exploit and AMADEUS, the inventory assets are evaluated using web scrapers or querying NVD. The outputs affect all potential vulnerabilities and exploits associated with the discovered system. If the system runs PostgreSQL version 16.4, the tool might find vulnerabilities like CVE-2020-0985 (REFRESH MATERIALIZED VIEW CONCURRENTLY executes arbitrary SQL). Then, AMADEUS-Exploit generates feature models (FMs) including all possible combinations of affected configurations, versions of product (CPEs) and exploits. As a result, the cyber team can verify that PostgreSQL version 16.4 is vulnerable under specific configurations, and no exploit currently exists for CVE-2020-0985. The reasoning mechanism prioritized CVEs with exploits that can affect critical assets, such as Adobe Commerce versions 2.4.3-p1 and 2.3.7-p2 affected by CVE-2022-24086, with related exploits already available, and focusing efforts to apply necessary preventive measures.

Thus, the earlier techniques provided specific examples of how to use feature models for vulnerability identification through capturing the global picture and the dependence between security information elements. We will then go over another basket of techniques related to the AI-based approach.

4.4. AI-Based Approach

This category focuses on the use of artificial intelligence (AI) technologies in identifying, classifying, and prioritizing vulnerabilities in software systems. It combines the use of one or multiple AI models (machine learning, deep learning, and LLM), as shown in Figure 22, to provide advanced techniques for finding and fixing vulnerabilities faster and more accurately than traditional methods, thereby strengthening the overall security posture of software systems. There are many contributions accomplished regarding this topic, as detailed in the following subsections:

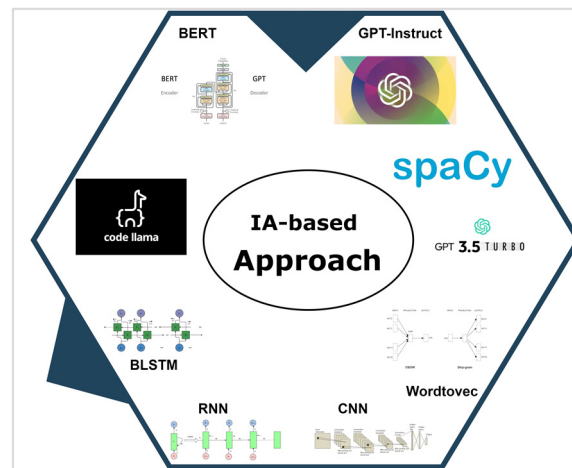


Figure 22. Features of AI-based approach.

4.4.1. AI-Based Approach Methods Description

Method Based on BLSTM

In 2018, the authors published a new method called Vulnerability Deep Pecker (VulDeePecker). The purpose of the method is to integrate deep learning into the software vulnerability detection process [109]. Based on this work, VulDeePecker automates vulnerability discovery by lowering the reliance on human experts, reducing false positive and negative rates, and enhancing detection accuracy. As one of the first attempts to integrate deep learning into vulnerability detection, VulDeePecker operates in two phases: learning and detection. The learning phase uses a large number of code gadgets classified as vulnerable or not for training the Bidirectional Long Short-Term Memory (BLSTM) network using Theano [98] and Keras [110]. In addition, the detection phase uses the trained BLSTM network to identify vulnerabilities in the program code. In addition, the target code is systematically converted to a vector using the word2vec tool [65], making it a suitable input for the BLSTM. Additionally, this model uses two datasets (NVD [36] and SARD [90]) to learn and detect vulnerability patterns from these vectorized code gadgets. Finally, preserving semantic relationships between programs, finer granularity representation of the code, and model suitability for the vulnerability detection context are VulDeePecker's guiding principles for employing deep learning in vulnerability detection. According to the experimental results, VulDeePecker achieved much fewer false negatives than other methods.

Method Based on NER

Another contribution proposed a new solution to address security issues during software development [10]. In this context, Dependency Vulnerability Management (DVM) technologies automate software composition analysis (SCA) to match known vulnerabilities (CVEs) with used software components. It was observed that there was a time lag between the first CVE disclosure and the addition of CPEs to the vulnerability (the median time is almost 35 days). Automated technologies cannot immediately alert developers and users

to these vulnerabilities. As a result, software systems may become exposed to attacks during this time lag. This work proposes generating new CPEs from CVE summaries and identifying affected software by the published vulnerabilities using Named Entity Recognition (NER) [111,112]. The model reduces time lag and helps prevent “one-day” vulnerabilities using DVM to immediately estimate CPEs associated with a CVE.

The workflow begins with gathering CVE IDs, summaries, and potential CPEs from NVD followed by the Feature Engineering (FE) stage, which includes four steps: (i) Character Level Features enable learning security-related semantics using one-dimensional convolution (CNN-layer) [111]; (ii) Word Level Embeddings convert each word into a 50-, 100-, 200-, or 300-dimensional numerical vector reflecting the semantic content of the word using glove embeddings; (iii) Word Level Case Features contribute to ascertaining the label of the particular word; and (iv) Security Lexicon uses NVD information to create a glossary of frequently used lexicon linked to CPE. Next, outputs from the FE stage are used by the Bidirectional Long-Short-Term Memory (BLSTM) network to capture a word’s context in both forward and backward directions [113]. Then, the Conditional Random Field (CRF) is used to forecast the word label sequence by assigning a class to each word [101]. Following the training and optimization stages, models assist the prompt detection and remediation of vulnerabilities in DVM. They provide real-time CPE estimations for newly discovered CVEs with minimal latency.

Method Based on ML

In the same field of research, this method suggested a recommender system for tracking vulnerabilities that addresses the matching issue between public notifications from VBDs and the potentially vulnerable products in an enterprise information system [114]. This method provides a shortlist of candidate matches for human verification. The pipeline of this method comprises three steps: (S1) Based on the target system’s asset inventory data, the method uses NLP with the SpaCy [115] library to extract word vectors. These are converted to vectors by Word2Vec [65] to represent the most relevant semantic similarity followed by normalization to discard unnecessary symbols. (S2) Fuzzy matching integrates cosine similarity [116,117] to measure the similarity between vendor and product names of inventory packages and NVD [36]. (S3) The final step is related to machine learning, and uses a random forest classifier with a Gini impurity measure, to classify the candidate CPEs by confidence levels (highest, highest, medium, lowest, lowest, reject) and classification levels (vendor or product).

Method Based on Looking-Back-Enabled Machine Learning

Regarding the IoT environment, authors recently became prime targets for cyberattacks owing to the rapid growth of IoT devices [118]. Digital transformation is integrated into our daily activities, including multiple smart devices. To face this challenge, this study proposes an architecture for detecting and mitigating DoS/DDoS attacks for IoT. By analyzing the UDP, TCP, and HTTP packets employed in the attack, the detection component combines multiple basic classifiers with the Looking-Back concept (integrating historic attacks data) for identifying subcategories of attacks. Then, the second component of this architecture is responsible for addressing mitigation countermeasures by denying or rate-limiting certain types of traffic.

Method Based on Inconsistency Measurement

Another study in the realm of vulnerability management based on an AI method was presented by the authors for this purpose [11]. This study aims to show the inconsistencies and inaccuracies in certain VDBs, both within and across different VDBs. These findings help cyber specialists identify susceptible software and reduce false positives and negatives. The proposed method, VERNIER (VulnERable Software Name Inconsistency MEasuRement Method), notifies cyber teams of inconsistencies and inaccurate software names to mitigate associated vulnerabilities. After extracting unstructured software names from Chinese and

English VDBs using a tailored Named Entity Recognition (NER) model, VERNIER measures software name inconsistency from three perspectives: measurement level (character and semantics), categories (mismatching, overclaiming, underclaiming, and overlapping), and VDBs (across NVD and eight other DBs, and inside NVD). The findings reveal prevalence inconsistencies between multiple databases (matching level: 20.3% for character and 43.3% for semantic) and within the same database, especially between structured and unstructured software names.

To address these issues, VERNIER suggests a tool that identifies the wrong software names using a reward-punishment matrix. The tool aggregates data from various VDBs, performs pairwise comparison using a reward-punishment system for correct, incorrect, or missing software names, constructs a reward-punishment matrix, applies a weighting system to assign the importance to different databases, and generates alerts for evaluation and correction.

Method Based on Active Learning

The Blockchain combines cryptography and distributed deployment technologies with peer-to-peer (P2P) networks. This system includes smart contracts, a technology implemented on the Blockchain that is considered a critical component. Statistics show that over 44% of attacks target smart contracts, causing significant losses [119,120]. Many vulnerability detection approaches exist today (static, dynamic, and ML methods), but they suffer from significant drawbacks due to a lack of data labeling. To address this issue, ASSBert is proposed [121]. It is a framework using a training dataset expanded by active learning (manual annotation) [122] and semi-supervised learning (predicting the labels of the unlabeled data) [123–125] to train model Bert [126]. The ASSBert pipeline begins with data preprocessing (cleaning and formatting), followed by feature extraction (tokenization, updating the BERT model, padding checking), and an active learning module to select the most uncertain samples for manual labeling and model creation. After evaluating the uncertainty, a semi-supervised learning module predicts labels for high-confidence samples, followed by iterative training and improvement of the BERT model using manual and pseudo-labeled data.

Method Based on Repository-Level Evaluation System

This new study has recently introduced a novel technique called VulEval [127]. It aims to reduce the impact of insecure code in software engineering [128]. The method discovers vulnerabilities at the granularity of individual functions or files (inter-procedure) and across multiple files or repositories (inter-procedure) and predicts relevant dependencies related to vulnerabilities. To achieve this, repository-level data and contextual dependencies are incorporated into VulEval to overcome some limitations of existing vulnerability detection techniques. To do this, VulEval starts with data processing by gathering CVE entries, vulnerability patches, and C/C++ programming language types.

There are three assessment tasks in VulEval. Initially, it predicts if a source code fragment has a vulnerability. Second, using the (Callee and Caller) process, VulEval determines the vulnerability level between each possible dependence (retrieved from the call graph) and the input code snippet. Third, VulEval utilizes the “Detector” to integrate dependencies found in the second task and determine if the input (target function) is an inter-procedural vulnerability. This last task uses two open-source LLMs: LLaMA [129] and CodeLlama [130], as well as two closed-source LLMs: GPT-3.5-turbo and GPT-3.5-instruct (ChatGPT) [131], developed by OpenAI. The findings show that ChatGPT performs better in all empirical assessments. In addition, lexical-based methods are more successful than semantic ones in detecting dependencies, and incorporating vulnerability information at the repository level enhances model performance.

Method Based on Gradient Boosting Machine (GBM) and Lasso Regression

To reduce the severe risk from ransomware targeting IIoT (Industrial Internet of Things), running on ZephyrOS, a combination methodology is proposed based on GBM and Lasso Regression [132]. The hybrid solution used GBM to leverage large datasets to build decision trees and predict unusual patterns in SCADA (Supervisory Control and Data Acquisition) systems. Then, Lasso Regression focused on preventing overfitting by zeroing down irrelevant features and focusing on critical predictors of ransomware activity. By continuously monitoring data streams, the proposed solution notifies unusual activities from file extensions and directory structure changes, network traffic, resource utilization, and file encryption behaviors. When the anomaly score exceeds, the system immediately responds by isolating the affected device from the network to prevent ransomware spread and other preventive measures. More details are summarized in Table 6.

4.4.2. Findings Analysis

An existing link between the two first methods lies in their shared use of deep learning techniques to improve vulnerability detection and automation processes. While VulDeePecker focuses on analyzing code gadgets through a BLSTM network, achieving a 93% accuracy rate, the method based on NER uses the same network's feature outputs for sequence labeling and feeds them into a CRF to automate CPE extraction, reconstructing CPEs in 67.44% of CVEs. The recommender system for tracking vulnerabilities integrated machine learning, NLP, and fuzzy matching to significantly narrow down the search for affected products in the NVD, achieving notable success of 40% for software and 48% for hardware vulnerabilities. Similarly, the method based on looking-back-enabled machine learning techniques employed the random forest classifier to detect and mitigate DoS/DDoS attacks in IoT systems with an impressive accuracy of 99.81%. These approaches leverage different machine learning models to enhance the efficiency and accuracy of the vulnerability assessment process.

VERNIER and ASSBert highlighted the importance of data accuracy in improving vulnerability detection. VERNIER identified, measured, and mitigated inconsistencies in software names from multiple major vulnerability databases using NER models. By integrating a correcting tool, such as a reward-punishment matrix, it uncovered and alerted incorrect software names to improve the vulnerability detection process, achieving a 99.5% accuracy and an F1 score of 95.1%. ASSBert incorporated the BERT model with active learning and semi-supervised learning, demonstrating a performance rate between 79% and 89%, depending on the dataset used during the learning phase.

Another noteworthy method, VulEval, employed a multi-model approach combining static code analysis, supervised machine learning, and large language models (LLMs). Experimental results recorded a precision of 69.78% with the PILOT method, although further exploration of repository-level vulnerability detection remains necessary. The last method incorporated a hybrid approach using double machine learning models (GMB and LR) to identify ransomware threats in (IIoT). The promising results indicated a detection accuracy of 92%. The use of multiple models, along with other feature sets and datasets, has been critical in enhancing learning phases and improving the accuracy of AI models in the vulnerability detection process.

Table 6. Collected methods related to AI based-approach.

Authors, Year	Used Method	Scope or Ecosystem	Limitations and Challenges	Attributes	Human Interaction (HI)	Prioritization	Scanning Mode
Li et al., 2018 [109]	RNN (BLSTM); Word2vec; Theano Keras.	IT	Dependence on source code to detect vulnerabilities while complied program remains a challenge; Applicability only in C/C++ and for one vulnerability type (library / API function calls); VulDeePecker does not provide control flow analysis, it only supports data flow analysis; Dependence on the quality of datasets used in model training; Converting code gadget variable length vector representations into fixed-length vectors; The vulnerability detection results depend only on one model; No features to identify the reason behind false positives and negatives results.	Datasets: NVD and SARD; Target programs; Code Gadgets (vector).	Yes, especially in learning phase when Labeling code gadgets.	No	Passive
Wareus et al., 2020 [10]	NER BLSTM CRF CNN.	IT	Intensive processing power and time are needed to train models; The F-measure, recall, and precision indicate signs of an overfitting, which requires further training and hyperparameter of used model; When dealing with multi-word labels, the model performs less well; Lexicon limitations affect the performance of the proposed model; Complex sentences or unseen words in CVE affect the context understanding (BLSTM and CRF); Dependency on the quality and quantity of NVD data (inconsistence, errors, data lack, rare labels, exposure delay, amount of training data); Multi-word labels present issues that single one and affect the performance of proposed model; A significant number of errors are produced, leading to incorrect predictions (both over- and under-predicting of labels).	Data: NVD CVE ID and CPEs; CoNLL-2003 dataset for NER; CVE summary.	Yes, to handle errors in labeling activities.	No	Passive

Table 6. Cont.

Authors, Year	Used Method	Scope or Ecosystem	Limitations and Challenges	Attributes	Human Interaction (HI)	Prioritization	Scanning Mode
Huff et al., 2021[114]	NLP: SpaCy and Word2Vec; Fuzzy matching: cosine similarity; ML: RF.	IT	Software naming conventions influence matching accuracy; Inventory and NVD discrepancies can affect fuzzy matching and NLP processes; Human confirmation of outcomes influences process flexibility; Large dependency on the quality of the training dataset; The system generates results with false positives and negatives; The performance might have an influence on a vast size of organization; CVE without CPE Metadata remains a significant data constraint.	NVD (CVE and CPE); Names of Software Packages installed within an organization; Dataset (https://github.com/pdhuff/cpe_recommender).	Yes, for reviewing the shortlisted candidate CPEs and confirming matches.	No	Passive
Mihoub et al., 2022 [118]	MLP, RNN, LSTM, KNN, DT, RF.	IoT	Lack of temporal relationships between DOS and DDOS attacks in the dataset used; Significantly time is required for training and testing phases, which impact quick detection;	Bot-IoT Dataset [133].	No	No	Passive
Sun et al., 2023 [11]	NER, RNN, LSTM, Neural network.	IT	The tool's efficacy is based on the data quality in the nine VDBs; The reward-punishment matrix may provide inaccurate or misleading outcomes; Computationally intensive may influence the tool performances in a large-scale context; The tool may struggle with unclear ambiguous case when software names are not clear or general; The manual verification method may be both time-consuming and labor-intensive.	CVE IDs from NVD, CVE, CNNVD, CNVD, ExploitDB, SecurityFocus, Openwall, (EDB), and SecurityFocus Forum.	Yes, to validate the alerts, use descriptions and data from all vulnerability databases.	No	Passive

Table 6. Cont.

Authors, Year	Used Method	Scope or Ecosystem	Limitations and Challenges	Attributes	Human Interaction (HI)	Prioritization	Scanning Mode
Sun et al., 2023 [119]	Regular expressions; ABI to encode or decode; SMT checker; Bert model; Classifier model; KL divergence; Maximization function ElBO(.); Measuring uncertainty H(.	Blockchain	Model dependence on the quality and quantity of labeled data used for training; Accumulation of training errors due to incorrect labels when using semi-supervised learning; More time-consuming and less efficient in active learning module; In practical applications, labeling all code data for vulnerability detection remains a complex activity; Possible complexity and computational resources in a large-scale environment.	Labeled Source Code Unlabeled Source Code Datsets: Smartbugs [134], SoliAudit [135], and SolidiFi [136].	Yes, for manually labeling activities.	No	Passive
VulEval Wel et al., 2024 [127]	CodeBERT; CodeT5; UniXcoder; LLaMA; CodeLlama; GPT-3.5-turbo; GPT-3.5-instruct.	IT	Focus solely C/C++ and not generalizing well to other programming languages; Dependence on predefined rules and patterns (time-consuming and labor-intensive); Quality of dataset used; The complex nature and scope of a project might impact the accuracy of inter-procedural vulnerability; The semantic-based approach is not very effective; Evaluation in software development environments; Challenges in compiled code version.	Dataset: PRIMEVUL Source code target, file and repository.	Yes, for the input and assess the output of the second task.	Yes, for vulnerability- related dependency prediction.	Passive
Tariq, 2024 [132]	GBM, Lasso Regression	IIoT/ZephyrOS	Issues to detect modern ransomware altering their signature dynamically; GBM and Lasso Regression can encounter compatibility issues with legacy systems; Training steps require extensive time to handle large datasets; Improper tuning of hyperparameters (overfitting) can influence the model detection capacities; Imbalanced datasets can affect the performance of the used model.	Datasets: RanSAP and IoT-23.	No	No	Active

This study explores the utility of AI algorithms to detect vulnerabilities in different ecosystems. The principle models integrated in vulnerability detection methods integrate RNN, BLSTM, NER, LSTM, fuzzy matching, NLP, GBM, among others. As seen previously, it is possible to combine multiple AI models to enhance the capacity to discover and assess vulnerabilities and weaknesses across diverse systems. The complexity lies in the dependence on dataset quality to improve model scores and reduce false negative and positive rates. Regarding the IoT environment, and based on findings extracted from Systematic Literature Review (SLR) [137], the Intrusion Detection Systems (IDSs) based on AI methods are particularly effective in detecting anomalies and intrusions. In this context, several ML, DL models, and hybrid methods are used to face the growth of cyberattacks, including but not limited to Neural Networks (NN), Convolutional Deep Learning (CDL), Extreme Gradient Boosting (XGBoost), RNN, and Fuzzy Pattern Tree (FPT).

In summary, the previously described approaches—similarity-based, graph-based, FM-based, and AI-based—all combine multiple methods and techniques for vulnerability identification processes. Each approach has its own challenges and issues, which will be discussed in the following section.

5. Challenges and Potential Solutions for Automating Vulnerability Detection

Automating vulnerability detection in cybersecurity is vital for keeping pace with the fast-evolving threat landscape. However, cybersecurity professionals face several practical challenges in implementing and maintaining automated systems. This section examines these challenges from a practitioner's perspective and proposes potential solutions.

5.1. Data Challenges

Cybersecurity professionals often deal with data from a variety of sources, including vulnerability databases, network logs, mobile devices, edge systems, telemetry, sensor data, and threat intelligence feeds. These data can vary significantly in format, accuracy, and detail. Inconsistent, low-quality, redundancy, noise, volatility state, lack of historical, heterogeneous sources, lack of labeling and incomplete data, device resource constraints, data fragmentation, and imbalanced data, among others, complicate the automation of vulnerability detection, leading to unreliable results [11,138–140].

Potential Solutions. To enhance data quality, cybersecurity teams can implement robust data preprocessing techniques. These include data cleaning, normalization, and enrichment processes to standardize and improve the quality of the data before they are fed into automated systems. Moreover, adding middleware layers and distributed data aggregation, within IoT and mobile environments, unifies data from different communication protocols and collects data at the edge. Additionally, integrating machine learning models can help predict and fill in missing data, improving the completeness and reliability of the datasets [9,141–143].

5.2. Cyber Risks Challenges

New technologies and high complexity emerge as new vulnerabilities, making systems outdated quickly. Moreover, new sophisticated cyberattacks using advanced strategies, such as advanced persistent threats (APT), outline real challenges for cyber teams in handling and reducing the nature of these cyber risks. Additionally, zero-day exploits harness organizations before they are found or fixed [144,145].

Potential Solutions. To address these issues, adopting automated platforms for continuous learning models helps us stay up to date with the latest vulnerabilities. Moreover, ongoing monitoring of assets for behavior-based analysis and immediate software patching are required to reduce the impact of zero-day vulnerabilities [146].

5.3. Infrastructure Challenges

This category of challenges is multifaceted owing to the diversity, complexity, and scale of modern environments. The target environments and protected systems face

multiple issues related to scalability and performance (cloud and distributed systems), diversity of components (not limited to legacy systems, IoT devices, edge computing), resource constraints (computational, storage, and battery resources), real-time detection and response (financial and industrial control systems (ICS)), lack of standardization (CPE, inconsistency vendor names, varying protocols), limited visibility in multi-tenant (cloud environments), among others [147,148].

Potential Solutions. To bypass the previous issues, it is recommended to adopt real-time monitoring solutions (like cloud-native monitoring, lightweight detection systems, behavioral anomaly analysis), lightweight models and offloading computational tasks for IoT devices (edge computing, firmware updates and patch automation, and protocol-aware detection), integrate AI models of vulnerability detection associated with the continuous training process, and federated learning for IoT devices and edge computing, among others, to ease the automation of vulnerability detection and reduce its impact [149–152].

5.4. False Positives and Negatives Challenges

Automated systems are prone to generating false positives, which can overwhelm cybersecurity teams with unnecessary alerts, and false negatives, where real threats go undetected. Both scenarios are problematic: false positives can lead to alert overload, causing critical warnings to be missed, while false negatives can leave systems vulnerable to exploitation. Cybersecurity professionals need to fine-tune these systems to balance sensitivity and specificity.

Potential Solutions. To address this challenge, many ways, among others, are proposed based on advanced and real-world applications of ML [153], heuristic-based detection and behavioral analysis [154], unsupervised learning techniques leveraging deviation from normal behavior [155], using multiple machine learning models in tandem, advanced data representation by including graph-based representations (control flow, syntax, and semantic graphs) [156], and contextual embeddings [157], which allow the model to understand more complex relationships.

Thus, multiple challenges are raised facing the advanced technologies and the complexity of existing systems. Next, we will delve into the discussion section to underscore certain points.

6. Discussion and Synthesis

The four previously discussed vulnerability approaches use diverse techniques and algorithms with various inputs and outputs. These approaches have advantages and drawbacks and aim to reduce the high rate of false positives and negatives. However, their effectiveness needs further improvement.

It is worth noting that the AI-based approach represents a trend in scientific research in vulnerability detection and cyber risk prediction. This is based on the analysis of the six methods (Table 7) and observations from the connected papers (Table 8). This fact is supported by the newly published methods and their promising results. Moreover, this category depends on generating reliable datasets, which is labor-intensive and time-consuming. In this context, we found that converting model inputs to vector form using various techniques is advisable, as detailed in Section 4. This involves preserving the input's lexical, syntactical, and semantic states. Concerning ML models, Logistic Regression (LR), Gaussian Naive Bayes (GNB), Support Vector Machine (SVM), Decision Tree (DT), Deep Belief Network (DBN), Extra Trees Classifier (ETC), Voting Classifier (VC), Random Forest (RF), K Nearest Neighbor (KNN), Bagging Classifier (BC), Gradient Boosting (GB), AdaBoost Classifier (AC), XGBoost (XB), among others, are used alone or combined in many studies related to vulnerability detection solutions with various levels of accuracy and depending on the nature and the attack complexity. Hybrid ML solutions are mentioned in the literature review; we notice that ML techniques are combined with either a layer of statistical criteria, feature selection based on TFIDF, the Looking-Back concept, or other ML models. Regarding the DL models, several studies proposed attack detection solutions

including one or multiple models like Convolutional Neural Networks (CNNs), Recurrent Neural Networks (RNNs), Long Short-Term Memory (LSTM), Feedforward Neural Networks (FNN), Gated Recurrent Units (GRU), Variational Autoencoders (VAE), Graph Neural Networks (GNN), Autoencoders (AEs), Deep Belief Networks (DBNs), Generative Adversarial Networks (GANs), Deep Reinforcement Learning (DRL), among others. The DL can improve their performance if they are associated with another layer of Metaheuristic Algorithms (MAs) [158]. These MAs contribute to optimizing and tuning DL models and improving their effectiveness. This combination enhances the detection and response of cyberattack capabilities [159]. Moreover, many studies explored the advantages of AI/XAI in cyberattacks detection. Proposed studies including these features are referenced in “connected papers”, the third column of Table 7. Finding the ideal AI model with accurate hyperparameters is still challenging and requires further study and real-world evaluation to reduce hallucination issues and errors.

Table 7. Connected papers, from 2016 to 2024, related to vulnerability detection per approach.

Category	Domain	Features Trend	Connected Papers
IA-based approach	Vulnerability detection based on Deep and Machine Learning.	Deep and Machine Learning (CNN, DNN, RNN, LSTM, BLSTM, FNN, VAE, GNN, AEs, GANs, DRL, RF, LR, DT, ETC, VC, BC, AC, GB, XB, GRU, DBN, MLP, K-fold Stacking Model (RF, GNB, KNN, SVM, GB, AdaLR, ADA, SVC, RFC, XAI)).	[62,141,143,160–185]
	Vulnerability detection based on OpenAI-Metaheuristic algorithms.	Large Language Model (LLM, GPT-2, GPT-3, GPT-3.5, GPT-4, Llama, PaLM2)-Metaheuristic algorithms (Genetic Algorithm (GA), Genetic Programming (GP), Particle Swarm Optimization (PSO), Teaching–Learning-Based Optimization (TLBO), among others).	[186–208]
Feature model-based approach	Vulnerability feature model-mapping, dependencies, and correlations of system components.	Cybersecurity knowledge base, reverse engineering, metamodel, Algorithms FM (SubFM/Vendor, SubFM/RC and SubFM/Tree), FaMaPy.	[209–223]
Graph-based approach	Vulnerability detection based on graph structure information related to target input and strengthened by certain AI techniques.	AST-PDG-CPG-Gremlin graph-EDG-Graph-based analytic-Graph traversal-Threat knowledge graph -GNN-SPG-LLM and AI model.	[58,95,156,224–245]
Matching-based approach	Vulnerability detection based on string-matching algorithm and AI models.	RE–Levenshtein edit distance–TF-IDF–Ratcliff/Obershelp–fuzzy matching; AST–Hash algorithms–Jaro–Winkler–GPT models.	[246–260]

In terms of the feature model-based approach, the main objective is to utilize this method in order to detect vulnerabilities. This is performed by providing a comprehensive overview of the system components, which is presented in graphical and textual notations. Additionally, it allows for the current dependencies and potential correlations between all sub-elements of FM (CVE, product, RC, CPE, etc.) to be identified. This capability helps identify and analyze the target system in depth, diagnose possible security flaws, and mitigate cyber risks. For a more accurate FM, it is recommended to check the relevance of the security data source and the asset inventory. It remains crucial to note that large configuration variability increases FM complexity, requiring human interaction for maintenance, which is time-consuming and prone to errors.

Table 8. Summary of observed limits related to four studied approaches.

Vulnerability Detection Approaches	Summary of Limits and Drawbacks Related to the Four Aforementioned Approaches.
A matching-based approach	- VDBs, which store and publish all multiform security events, contain multiple issues related to inconsistent software products, missing metadata, especially CPE, lack of synchronization between CPE dictionary and CPE/CVE; - String-similarity algorithms generate errors during the process matching, which increases false positives and negatives; - Assets inventory do not incorporate a complete CPE list product; - Configuration variability and instable product naming over the time impact the accuracy of results; - Vulnerability zero day is still a rising crucial issue; - Difficulty to perform a similarity matching between product having the same semantic and different syntax; - The string-matching process is labor-intensive and significantly computational; - Inaccurate results when using GPT models.
A graph-based approach	- Building an accurate graph to represent all slices of source code represents a challenge, specifically in a large complex ecosystem; - Both the quality of data for the training model and building graph is so important to avoid errors and under-exploitation; - Certain techniques center their study on identifying weaknesses in a particular programming language; - The graph-based approach could include an excessive amount of duplicate data unrelated to the vulnerabilities; - The graph-based approach, which seeks to improve the brute-force method, is resource-intensive and has to be optimized in order to perform better; - Some techniques do not employ steps to isolate the compromised network segment in order to prevent the spread of threats; - The white-box analysis makes it more difficult to produce a precise model representation, particularly in ICS.
A feature model-based approach	- Mapping cartography or assets discovered as a feature model input is labor-intensive and prone to errors; - Errors occur during generating global FM when assessing huge system configurations, extreme complexity and variability of system components; - The relevancy of security events released by VDBs is essential to FM's accuracy; - The accuracy of the FMs may be impacted by discrepancies and a lack of pertinent data; - To keep the FM up to date, maintainability and real-time upgrades are necessary; - Human intervention is necessary for asset scanning, process analysis, FM update, and results exploitation.
An AI-based approach	- The quality of the datasets continues to be a factor in how well AI models operate; - There are challenges with the training process as it is time-consuming and labor-intensive; - Choosing the appropriate model for the context's method still poses a challenge in order to lower the rate of false positives and negatives; - Research elucidating the cause of false positives and negative outcomes is lacking; - Incorrect predictions occur when some models deal with lexical features; - The discrepancies and inconsistencies of published VDBs have an effect on the dataset quality; - Using Large Language Models (LLMs) to accurately identify software vulnerabilities without generating false positives is still challenging; - Not all the vulnerabilities can be discovered by employing the methods that have been suggested.

In addition, the graph-based approach describes target components, their relationships, and potential dependencies (asset inventory, snippet code, software system, CVE, CPE, etc.). Other AI models can thoroughly analyze every system component for vulnerabilities. The training phase of models can use data from the graph representation (data, flow, and control). To prevent a high rate of false positives and negatives, it is helpful to build an accurate representation and reflect an exact dependence. Furthermore, accurate and thorough representation helps pretrain the model, improving performance with LLM or other AI models.

Regarding the similarity matching approach, we examined multiple methods from 2016 to 2024, as demonstrated in Section 4. This approach integrates several string-matching algorithms. They evaluate the similarity between the asset source (CPE) and target (CPE dictionary or CVE/CPE) to identify potential vulnerabilities. These findings cover all previous methods of this approach, except for the last one, which uses OpenAI prompts to search for security events. We identified three issues with this approach: (i) the matching algorithm often generates errors and misses specific values; (ii) VDBs present challenges that affect the matching process; and (iii) the asset inventory source value is often inaccurate.

Further details are in Tables 7 and 8. We include a literature scope of connected papers per approach, incorporating the same topic using different methods.

As a result, the assessment of the capacity of vulnerability detection methods revealed several shortcomings and issues in several aspects. This study has to be thoroughly examined in order to provide effective answers and practical recommendations that will improve the performance of the earlier techniques and prevent errors.

7. Conclusions and Future Work

Organizations are implementing several tactics and procedures to reduce the damage and protect inventory assets due to the rise in cyberattacks and new vulnerabilities. This study assesses literature from 2016 to 2024, focusing on current approaches for vulnerability identification. It presents several methods for vulnerability detection. The analysis study highlights limitations and drawbacks of current methodologies, classifies them into four approaches, and provides significant insights. Additionally, a comparative analysis of many vulnerability databases was conducted, emphasizing the crucial role of these VDBs in the risk management process and dependence on published data. In addition, the literature review also highlights scientific contributions related to our theme, categorized by approaches for future investigation.

To further reduce the false positive and negative rate and design efficient methodologies in the vulnerability detection process, we plan to continue our research work throughout these three future work directions:

- Examine the possibility to build an automated system to collect security events in real time from external sources and perform preprocessing data;
- Build a new vulnerability dataset for well-trained and learning AI models;
- Develop an AI model combined with metaheuristics algorithms or other layers to enhance model capacities in vulnerability detection methods, within different ecosystems.

Author Contributions: Conceptualization, K.B. and N.A.A.; methodology, K.B., N.A.A. and A.Z.F.; software, K.B. and D.M.; validation, K.B., N.A.A., Y.E.B.E.I. and A.Z.F.; formal analysis, K.B., N.A.A. and B.S.; investigation, K.B., N.A.A. and D.M.; resources, K.B. and B.S.; data curation, K.B. and D.M.; writing—original draft preparation, K.B. and N.A.A.; writing—review and editing, K.B., N.A.A., A.Z.F. and D.M.; visualization, K.B. and N.A.A.; supervision, N.A.A. and Y.E.B.E.I.; project administration, Y.E.B.E.I.; funding acquisition, B.S. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Data Availability Statement: The data that support the findings of this study are provided by ACG Cybersecurity (<https://acgcybersecurity.fr/>, accessed on 5 August 2024).

Acknowledgments: We acknowledge the collaborative efforts of the Laboratory of Engineering Sciences, National School of Applied Sciences, Ibn Tofail University, Kenitra 14000, Morocco, and ACG Cybersecurity, for their contributions to the research design and data analysis. Special thanks to the R&D teams for their insightful discussions and feedback throughout this research study.

Conflicts of Interest: The authors declare that they have no financial or personal conflicts of interest that could have influenced the work reported in this manuscript. All authors provided materials and contributed effectively to support this research without influencing this study.

References

1. Top Cybersecurity Statistics for 2024. Available online: <https://www.cobalt.io/blog/cybersecurity-statistics-2024> (accessed on 21 July 2024).
2. Gartner Identifies Three Factors Influencing Growth in Security Spending. Available online: <https://www.gartner.com/en/newsroom/press-releases/2022-10-13-gartner-identifies-three-factors-influencing-growth-i> (accessed on 18 April 2024).
3. Rossella, M.; Apostolos, M.; ENISA. Foresight Cybersecurity Threats for 2030—Update. *Creat. Commons Attrib. 40 Int. CC 40* **2024**, 7–12. Available online: <https://data.europa.eu/doi/10.2824/349493> (accessed on 31 July 2024).
4. Pochmara, J.; Świetlicka, A. Cybersecurity of Industrial Systems—A 2023 Report. *Electronics* **2024**, *13*, 1191. [\[CrossRef\]](#)
5. Ushakov, R.; Doynikova, E.; Novikova, E.; Kotenko, I. CPE and CVE Based Technique for Software Security Risk Assessment. In Proceedings of the 2021 11th IEEE International Conference on Intelligent Data Acquisition and Advanced Computing Systems: Technology and Applications (IDAACS), Cracow, Poland, 22–25 September 2021; IEEE: Piscataway, NJ, USA, 2021; pp. 353–356.
6. Kharat, P.P.; Chawan, P.M. Vulnerability Management System. *Int. Res. J. Eng. Technol.* **2022**, *9*, 976–981.
7. Computer Security Division, I.T.L. Security Content Automation Protocol | CSRC | CSRC. Available online: <https://csrc.nist.gov/projects/security-content-automation-protocol> (accessed on 18 April 2024).
8. Vladimir, D. CPE Ontology. 2021. Available online: <https://ceur-ws.org/Vol-2933/paper30.pdf> (accessed on 31 July 2024).
9. Sanguino, L.A.B.; Uetz, R. Software Vulnerability Analysis Using CPE and CVE. *arXiv* **2017**, arXiv:1705.05347.
10. Wåreus, E.; Hell, M. Automated CPE Labeling of CVE Summaries with Machine Learning. In *Detection of Intrusions and Malware, and Vulnerability Assessment*; Maurice, C., Bilge, L., Stringhini, G., Neves, N., Eds.; Lecture Notes in Computer Science; Springer International Publishing: Cham, Switzerland, 2020; Volume 12223, pp. 3–22, ISBN 978-3-030-52682-5.
11. Sun, H.; Ou, G.; Zheng, Z.; Liao, L.; Wang, H.; Zhang, Y. Inconsistent Measurement and Incorrect Detection of Software Names in Security Vulnerability Reports. *Comput. Secur.* **2023**, *135*, 103477. [\[CrossRef\]](#)
12. Tranfield, D.; Denyer, D.; Smart, P. Towards a Methodology for Developing Evidence-Informed Management Knowledge by Means of Systematic Review. *Br. J. Manag.* **2003**, *14*, 207–222. [\[CrossRef\]](#)
13. Swanson, M.; Hash, J.; Bowen, P. *Guide for Developing Security Plans for Federal Information Systems*; National Institute of Standards and Technology: Gaithersburg, MD, USA, 2006; p. 47.
14. Newhouse, W. *Multifactor Authentication for E-Commerce*; National Institute of Standards and Technology: Gaithersburg, MD, USA, 2019; p. 24.
15. *ISO/IEC 27005; Information Security, Cybersecurity and Privacy Protection—Recommendations for the Management of Risks Related to Information Security*. ISO: Geneva, Switzerland, 2022.
16. Joint Task Force Transformation Initiative. *Risk Management Framework for Information Systems and Organizations: A System Life Cycle Approach for Security and Privacy*; National Institute of Standards and Technology: Gaithersburg, MD, USA, 2018; pp. 21–23.
17. Isniah, S.; Hardi Purba, H.; Debora, F. Plan Do Check Action (PDCA) Method: Literature Review and Research Issues. *J. Sist. Dan Manaj. Ind.* **2020**, *4*, 72–81. [\[CrossRef\]](#)
18. Joint Task Force Transformation Initiative. *Guide for Conducting Risk Assessments*; Department of Commerce, National Institute of Standards and Technology: Gaithersburg, MD, USA, 2012; p. 53.
19. Stine, K.; Kissel, R.; Barker, W.C.; Fahlsing, J.; Gulick, J. Volume I: Guide for Mapping Types of Information and Information Systems to Security Categories. *Spec. Publ. 800-60 Revis. 1* **2008**, *1*, 53. [\[CrossRef\]](#)
20. Ross, R.; Pillitteri, V.; Graubart, R.; Bodeau, D.; McQuaid, R. *Developing Cyber-Resilient Systems: A Systems Security Engineering Approach*; National Institute of Standards and Technology (U.S.): Gaithersburg, MD, USA, 2021; pp. 17–18+91–92.
21. National Institute of Standards and Technology. *Framework for Improving Critical Infrastructure Cybersecurity, Version 1.1*; National Institute of Standards and Technology: Gaithersburg, MD, USA, 2018. [\[CrossRef\]](#)
22. LeMay, E.; Scarfone, K.; Mell, P. *The Common Misuse Scoring System (CMSS): Metrics for Software Feature Misuse Vulnerabilities*; National Institute of Standards and Technology: Gaithersburg, MD, USA, 2012; pp. 16–17+20.
23. Nieves, M.; Dempsey, K.; Pillitteri, V.Y. *An Introduction to Information Security*; National Institute of Standards and Technology: Gaithersburg, MD, USA, 2017; pp. 12–13.
24. Cichonski, P.; Millar, T.; Grance, T.; Scarfone, K. *Computer Security Incident Handling Guide: Recommendations of the National Institute of Standards and Technology*; National Institute of Standards and Technology: Gaithersburg, MD, USA, 2012; pp. 34–35.
25. Franklin, J.; Wergin, C.; Booth, H. *CVSS Implementation Guidance*; National Institute of Standards and Technology: Gaithersburg, MD, USA, 2014; p. 16.
26. *ISO/IEC 27001 ISO/IEC; Information Security, Cybersecurity and Privacy Protection—Information Security Management Systems—Requirements*. ISO: Geneva, Switzerland, 2022.
27. *ISO/IEC 27032; Cybersecurity—Guidelines for Internet Security*. ISO: Geneva, Switzerland, 2023.
28. Johnson, C.S.; Badger, M.L.; Waltermire, D.A.; Snyder, J.; Skorupka, C. *Guide to Cyber Threat Information Sharing*; National Institute of Standards and Technology: Gaithersburg, MD, USA, 2016; p. 10.
29. Dempsey, K.; Eavy, P.; Moore, G. *Automation Support for Security Control Assessments. Volume 1: Overview*; National Institute of Standards and Technology: Gaithersburg, MD, USA, 2017; p. NIST IR 8011-1. [\[CrossRef\]](#)
30. Cheikes, B.A.; Waltermire, D.; Scarfone, K. *Common Platform Enumeration: Naming Specification Version 2.3*; National Institute of Standards and Technology: Gaithersburg, MD, USA, 2011; p. NIST IR 7695. [\[CrossRef\]](#)

31. Waltermire, D.; Cichonski, P.; Scarfone, K. *Common Platform Enumeration: Applicability Language Specification Version 2.3*; National Institute of Standards and Technology: Gaithersburg, MD, USA, 2011; p. NIST IR 7698. [CrossRef]
32. Phillips, A.; Davis, M. *Tags for Identifying Languages*; Internet Engineering Task Force: Fremont, CA, USA, 2009. [CrossRef]
33. CPE—Common Platform Enumeration: CPE Specifications. Available online: <https://cpe.mitre.org/specification/> (accessed on 21 April 2024).
34. Solving Problems for a Safer World | MITRE. Available online: <https://www.mitre.org/> (accessed on 13 July 2024).
35. Home Page | CISA. Available online: <https://www.cisa.gov/> (accessed on 13 July 2024).
36. NVD—Home. Available online: <https://nvd.nist.gov/> (accessed on 22 April 2024).
37. CWE—About CWE. Available online: <https://cwe.mitre.org/about/index.html> (accessed on 22 April 2024).
38. CVSS v4.0 Specification Document. Available online: <https://www.first.org/cvss/specification-document> (accessed on 20 April 2024).
39. Liu, Q.; Zhang, Y. VRSS: A New System for Rating and Scoring Vulnerabilities. *Comput. Commun.* **2011**, *34*, 264–273. [CrossRef]
40. Spanos, G.; Sioziou, A.; Angelis, L. WIVSS: A New Methodology for Scoring Information Systems Vulnerabilities. In Proceedings of the 17th Panhellenic Conference on Informatics, Thessaloniki, Greece, 19–21 September 2013; ACM: New York, NY, USA, 2013; pp. 83–90. [CrossRef]
41. Sharma, A.; Sabharwal, S.; Nagpal, S. A Hybrid Scoring System for Prioritization of Software Vulnerabilities. *Comput. Secur.* **2023**, *129*, 103256. [CrossRef]
42. Swanson, M.; Bowen, P.; Phillips, A.W.; Gallup, D.; Lynes, D. *Contingency Planning Guide for Federal Information Systems*; National Institute of Standards and Technology: Gaithersburg, MD, USA, 2010; p. 144.
43. NIST SP 800-53 Rev. 5; Joint Task Force Interagency Working Group Security and Privacy Controls for Information Systems and Organizations Revision 5. National Institute of Standards and Technology: Gaithersburg, MD, USA, 2020; 176–188+370.
44. GitHub: Let's Build from Here. Available online: <https://github.com/> (accessed on 8 July 2024).
45. Liu, B.; Shi, L.; Cai, Z.; Li, M. Software Vulnerability Discovery Techniques: A Survey. In Proceedings of the 2012 Fourth International Conference on Multimedia Information Networking and Security, Nanjing, China, 2–4 November 2012; IEEE: Piscataway, NJ, USA, 2012; pp. 152–156.
46. Gawron, M.; Cheng, F.; Meinel, C. PVD: Passive Vulnerability Detection. In Proceedings of the 2017 8th International Conference on Information and Communication Systems (ICICS), Irbid, Jordan, 4–6 April 2017; IEEE: Piscataway, NJ, USA, 2017; pp. 322–327.
47. Na, S.; Kim, T.; Kim, H. Service Identification of Internet-Connected Devices Based on Common Platform Enumeration. *J. Inf. Process. Syst.* **2018**, *14*, 740–750. [CrossRef]
48. Elbaz, C.; Rilling, L.; Morin, C. Automated Keyword Extraction from “One-Day” Vulnerabilities at Disclosure. In Proceedings of the NOMS 2020—2020 IEEE/IFIP Network Operations and Management Symposium, Budapest, Hungary, 20–24 April 2020; IEEE: Piscataway, NJ, USA, 2020; pp. 1–9.
49. Xu, Y.; Xu, Z.; Chen, B.; Song, F.; Liu, Y.; Liu, T. Patch Based Vulnerability Matching for Binary Programs. In Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis, Virtual, 18–22 July 2020; ACM: New York, NY, USA, 2020; pp. 376–387.
50. Zhao, Q.; Huang, C.; Dai, L. VULDEFF: Vulnerability Detection Method Based on Function Fingerprints and Code Differences. *Knowl.-Based Syst.* **2023**, *260*, 110139. [CrossRef]
51. Kornblum, J. Identifying Almost Identical Files Using Context Triggered Piecewise Hashing. *Digit. Investig.* **2006**, *3*, 91–97. [CrossRef]
52. McClanahan, K.; Li, Q. Towards Automatically Matching Security Advisories to CPEs: String Similarity-Based Vendor Matching. In Proceedings of the IEEE International Conference on Computing, Networking and Communications (ICNC)-Workshop on Computing, Networking and Communications, Big Island, HI, USA, 19–22 February 2024. [CrossRef]
53. McClanahan, K.; Elder, S.; Uwibambe, M.L.; Liu, Y.; Heng, R.; Li, Q. When ChatGPT Meets Vulnerability Management: The Good, the Bad, and the Ugly. In Proceedings of the IEEE International Conference on Computing, Networking and Communications (ICNC)-Workshop on Computing, Networking and Communications, Big Island, HI, USA, 19–22 February 2024. [CrossRef]
54. Gao, Z.; Zhang, C.; Liu, H.; Sun, W.; Tang, Z.; Jiang, L.; Chen, J.; Xie, Y. Faster and Better: Detecting Vulnerabilities in Linux-Based IoT Firmware with Optimized Reaching Definition Analysis. In Proceedings of the 2024 Network and Distributed System Security Symposium, San Diego, CA, USA, 26 February–1 March 2024; Internet Society: Reston, VA, USA, 2024. [CrossRef]
55. Wang, H.; Ye, G.; Tang, Z.; Tan, S.H.; Huang, S.; Fang, D.; Feng, Y.; Bian, L.; Wang, Z. Combining Graph-Based Learning with Automated Data Collection for Code Vulnerability Detection. *IEEE Trans. Inf. Forensics Secur.* **2021**, *16*, 1943–1958. [CrossRef]
56. Zhou, J.; Cui, G.; Hu, S.; Zhang, Z.; Yang, C.; Liu, Z.; Wang, L.; Li, C.; Sun, M. Graph Neural Networks: A Review of Methods and Applications. *AI Open* **2020**, *1*, 57–81. [CrossRef]
57. Noonan, R.E. An Algorithm for Generating Abstract Syntax Trees. *Comput. Lang.* **1985**, *10*, 225–236. [CrossRef]
58. Wen, X.-C.; Chen, Y.; Gao, C.; Zhang, H.; Zhang, J.M.; Liao, Q. Vulnerability Detection with Graph Simplification and Enhanced Graph Representation Learning. In Proceedings of the 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE), Melbourne, Australia, 17–19 May 2023; IEEE: Piscataway, NJ, USA, 2023; pp. 2275–2286.
59. Wu, Z.; Pan, S.; Chen, F.; Long, G.; Zhang, C. A Comprehensive Survey on Graph Neural Networks. *IEEE Trans. Neural Netw. Learn. Syst.* **2019**, *32*, 4–24. [CrossRef]
60. Cho, K.; van Merriënboer, B.; Gulcehre, C.; Bahdanau, D.; Bougares, F.; Schwenk, H.; Bengio, Y. Learning Phrase Representations Using RNN Encoder-Decoder for Statistical Machine Translation 2014. *arXiv* **2014**, arXiv:1406.1078.

61. Zheng, W.; Jiang, Y.; Su, X. Vu1SPG: Vulnerability Detection Based on Slice Property Graph Representation Learning. In Proceedings of the 2021 IEEE 32nd International Symposium on Software Reliability Engineering (ISSRE), Wuhan, China, 25–28 October 2021; IEEE: Piscataway, NJ, USA, 2021; pp. 457–467.
62. Li, Z.; Zou, D.; Xu, S.; Jin, H.; Zhu, Y.; Chen, Z. SySeVR: A Framework for Using Deep Learning to Detect Software Vulnerabilities. *IEEE Trans. Dependable Secur. Comput.* **2022**, *19*, 2244–2258. [\[CrossRef\]](#)
63. Ferrante, J. The Program Dependence Graph and Its Use in Optimization. *ACM Trans. Program. Lang. Syst.* **1987**, *9*, 319–349. [\[CrossRef\]](#)
64. Yamaguchi, F.; Golde, N.; Arp, D.; Rieck, K. Modeling and Discovering Vulnerabilities with Code Property Graphs. In Proceedings of the 2014 IEEE Symposium on Security and Privacy, San Jose, CA, USA, 18–21 May 2014; IEEE: Piscataway, NJ, USA, 2014; pp. 590–604.
65. Gensim: Topic Modelling for Humans. Available online: <https://radimrehurek.com/gensim/models/word2vec.html> (accessed on 1 June 2024).
66. Schlichtkrull, M.; Kipf, T.N.; Bloem, P.; van den Berg, R.; Titov, I.; Welling, M. Modeling Relational Data with Graph Convolutional Networks. In Proceedings of the Semantic Web: 15th International Conference, ESWC 2018, Heraklion, Crete, Greece, 3–7 June 2018. [\[CrossRef\]](#)
67. Tovarnak, D.; Sadlek, L.; Celeda, P. Graph-Based CPE Matching for Identification of Vulnerable Asset Configurations. In Proceedings of the 2021 IFIP/IEEE International Symposium on Integrated Network Management (IM), Virtual, 17–21 May 2021; pp. 986–991.
68. Longueira-Romero, Á.; Iglesias, R.; Flores, J.L.; Garitano, I. A Novel Model for Vulnerability Analysis through Enhanced Directed Graphs and Quantitative Metrics. *Sensors* **2022**, *22*, 2126. [\[CrossRef\]](#)
69. CAPEC—Common Attack Pattern Enumeration and Classification (CAPECTM). Available online: <https://capec.mitre.org/> (accessed on 4 May 2024).
70. ISA/IEC 62443; Industrial Communication Networks—Network and System Security Series of Standards. ISA: Durham, NC, USA, 2017.
71. Autonomy—Open-Source PLC Software. Available online: <https://autonomylogic.com/> (accessed on 7 June 2024).
72. Alves, T. Thiagorvalves/OpenPLC. Available online: <https://github.com/thiagorvalves/OpenPLC> (accessed on 7 June 2024).
73. Alves, T. Thiagorvalves/OpenPLC_v2. Available online: https://github.com/thiagorvalves/OpenPLC_v2 (accessed on 7 June 2024).
74. Alves, T. Thiagorvalves/OpenPLC_v3. Available online: https://github.com/thiagorvalves/OpenPLC_v3 (accessed on 7 June 2024).
75. Husák, M.; Khoury, J.; Klisura, Đ.; Bou-Harb, E. On the Provision of Network-Wide Cyber Situational Awareness via Graph-Based Analytics. In *Complex Computational Ecosystems*; Collet, P., Gardashova, L., El Zant, S., Abdulkarimova, U., Eds.; Lecture Notes in Computer Science; Springer Nature Switzerland: Cham, Switzerland, 2023; Volume 13927, pp. 167–179, ISBN 978-3-031-44354-1.
76. Jajodia, S.; Liu, P.; Swarup, V.; Wang, C. *Cyber Situational Awareness: Issues and Research*; Springer Science & Business Media: Berlin/Heidelberg, Germany, 2009; ISBN 978-1-4419-0140-8.
77. Jiang, C.; Coenen, F.; Zito, M. A Survey of Frequent Subgraph Mining Algorithms. *Knowl. Eng. Rev.* **2013**, *28*, 75–105. [\[CrossRef\]](#)
78. Brandes, U. A Faster Algorithm for Betweenness Centrality*. *J. Math. Sociol.* **2001**, *25*, 163–177. [\[CrossRef\]](#)
79. De, S.; Sodhi, R. A PMU Assisted Cyber Attack Resilient Framework against Power Systems Structural Vulnerabilities. *Electr. Power Syst. Res.* **2022**, *206*, 107805. [\[CrossRef\]](#)
80. Shi, Z.; Matyunin, N.; Graffi, K.; Starobinski, D. Uncovering CWE-CVE-CPE Relations with Threat Knowledge Graphs. *ACM Trans. Priv. Secur.* **2024**, *27*, 1–26. [\[CrossRef\]](#)
81. Bordes, A.; Usunier, N.; Garcia-Duran, A.; Weston, J.; Yakhnenko, O. Translating Embeddings for Modeling Multi-Relational Data. *Proc. 26th Int. Conf. Neural Inf. Process. Syst.* **2013**, *2*, 2787–2795.
82. Trouillon, T.; Welbl, J.; Riedel, S.; Gaussier, É.; Bouchard, G. Complex Embeddings for Simple Link Prediction. In Proceedings of the International Conference on Machine Learning, New York, NY, USA, 19–24 June 2016.
83. Yang, B.; Yih, W.; He, X.; Gao, J.; Deng, L. Embedding Entities and Relations for Learning and Inference in Knowledge Bases. *arXiv* **2014**. [\[CrossRef\]](#)
84. Lu, G.; Ju, X.; Chen, X.; Pei, W.; Cai, Z. GRACE: Empowering LLM-Based Software Vulnerability Detection with Graph Structure and in-Context Learning. *J. Syst. Softw.* **2024**, *212*, 112031. [\[CrossRef\]](#)
85. Wu, Y.; Zou, D.; Dou, S.; Yang, W.; Xu, D.; Jin, H. VulCNN: An Image-Inspired Scalable Vulnerability Detection System. In Proceedings of the 44th International Conference on Software Engineering, Pittsburgh, PA, USA, 21 May 2022; ACM: New York, NY, USA, 2022; pp. 2365–2376.
86. Salayma, M. Threat Modelling in Internet of Things (IoT) Environments Using Dynamic Attack Graphs. *Front. Internet Things* **2024**, *3*, 1306465. [\[CrossRef\]](#)
87. Neo4j—Plateforme de Données de Graphes. Available online: <https://neo4j.com/fr/> (accessed on 2 May 2024).
88. Project-Kb/MSR2019 at Main · SAP/Project-Kb. Available online: <https://github.com/SAP/project-kb/tree/main/MSR2019> (accessed on 17 May 2024).
89. SecretPatch SecretPatch/Dataset. Available online: <https://github.com/SecretPatch/Dataset> (accessed on 17 May 2024).
90. NIST Software Assurance Reference Dataset. Available online: <https://samate.nist.gov/SARD> (accessed on 14 May 2024).
91. Wang, Y.; Wang, W.; Joty, S.; Hoi, S.C.H. CodeT5: Identifier-Aware Unified Pre-Trained Encoder-Decoder Models for Code Understanding and Generation. *arXiv* **2021**, arXiv:2109.00859.

92. Belkina, A.C.; Ciccolella, C.O.; Anno, R.; Halpert, R.; Spidlen, J.; Snyder-Cappione, J.E. Automated Optimized Parameters for T-Distributed Stochastic Neighbor Embedding Improve Visualization and Analysis of Large Datasets. *Nat. Commun.* **2019**, *10*, 5415. [CrossRef]
93. Yang, G.; Chen, X.; Cao, J.; Xu, S.; Cui, Z.; Yu, C.; Liu, K. ComFormer: Code Comment Generation via Transformer and Fusion Method-Based Hybrid Code Representation. In Proceedings of the 2021 8th International Conference on Dependable Systems and Their Applications (DSA), Yinchuan, China, 11–12 September 2021. [CrossRef]
94. Chakraborty, S.; Krishna, R.; Ding, Y.; Ray, B. Deep Learning Based Vulnerability Detection: Are We There Yet? *IEEE Trans. Softw. Eng.* **2022**, *48*, 3280–3296. [CrossRef]
95. Zhou, Y.; Liu, S.; Siow, J.; Du, X.; Liu, Y. Devign: Effective Vulnerability Identification by Learning Comprehensive Program Semantics via Graph Neural Networks. *Conf. Neural Inf. Process. Syst.* **2019**. [CrossRef]
96. Fan, J.; Li, Y.; Wang, S.; Nguyen, T.N. A C/C++ Code Vulnerability Dataset with Code Changes and CVE Summaries. In Proceedings of the 17th International Conference on Mining Software Repositories, Seoul, Republic of Korea, 29 June 2020; ACM: New York, NY, USA, 2020; pp. 508–512.
97. Batory, D.; Benavides, D.; Ruiz-Cortés, A. Automated Analysis of Feature Models. *Commun. ACM* **2006**, *49*, 45–47. [CrossRef]
98. Batory, D. Feature Models, Grammars, and Propositional Formulas. In *Software Product Lines*; Obbink, H., Pohl, K., Eds.; Lecture Notes in Computer Science; Springer: Berlin/Heidelberg, Germany, 2005; Volume 3714, pp. 7–20, ISBN 978-3-540-28936-4.
99. Varela-Vaca, Á.J.; Gasca, R.M.; Ceballos, R.; Gómez-López, M.T.; Torres, P.B. CyberSPL: A Framework for the Verification of Cybersecurity Policy Compliance of System Configurations Using Software Product Lines. *Appl. Sci.* **2019**, *9*, 5364. [CrossRef]
100. Galindo, J.A.; Benavides, D.; Trinidad, P.; Gutiérrez-Fernández, A.-M.; Ruiz-Cortés, A. Automated Analysis of Feature Models: Quo Vadis? *Computing* **2019**, *101*, 387–433. [CrossRef]
101. Brailsford, S.C.; Potts, C.N.; Smith, B.M. Constraint Satisfaction Problems: Algorithms and Applications. *Eur. J. Oper. Res.* **1999**, *119*, 557–581. [CrossRef]
102. Prud'homme, C.; Fages, J.-G.; Lorca, X. Choco-Solver. Available online: <https://choco-solver.org/> (accessed on 5 June 2024).
103. Benavides, D.; Trinidad, P.; Ruiz-Cortés, A.; Segura, S. FaMa. In *Systems and Software Variability Management: Concepts, Tools and Experiences*; Capilla, R., Bosch, J., Kang, K.-C., Eds.; Springer: Berlin/Heidelberg, Germany, 2013; pp. 163–171, ISBN 978-3-642-36583-6.
104. Kenner, A.; Dassow, S.; Lausberger, C.; Krüger, J.; Leich, T. Using Variability Modeling to Support Security Evaluations: Virtualizing the Right Attack Scenarios. In Proceedings of the 14th International Working Conference on Variability Modelling of Software-Intensive Systems, Magdeburg, Germany, 5 February 2020; ACM: New York, NY, USA, 2020; pp. 1–9.
105. Maynor, D. *Metasploit Toolkit for Penetration Testing, Exploit Development, and Vulnerability Research*; Maynor, D., Mookhey, K.K., Eds.; Syngress: Burlington, MA, USA, 2007; pp. vii–ix, ISBN 978-1-59749-074-0.
106. Varela-Vaca, Á.J.; Gasca, R.M.; Carmona-Fombella, J.A.; Gómez-López, M.T. AMADEUS: Towards the AutoMated security testing. In Proceedings of the 24th ACM Conference on Systems and Software Product Line, Montreal, QC, Canada, 19 October 2020; ACM: New York, NY, USA, 2020; Volume A, pp. 1–12.
107. Varela-Vaca, Á.J.; Borrego, D.; Gómez-López, M.T.; Gasca, R.M.; Márquez, A.G. Feature Models to Boost the Vulnerability Management Process. *J. Syst. Softw.* **2023**, *195*, 111541. [CrossRef]
108. Galindo, J.A.; Benavides, D. A Python Framework for the Automated Analysis of Feature Models: A First Step to Integrate Community Efforts. In Proceedings of the 24th ACM International Systems and Software Product Line Conference, Montreal, QC, Canada, 19 October 2020; ACM: New York, NY, USA, 2020; Volume B, pp. 52–55.
109. Li, Z.; Zou, D.; Xu, S.; Ou, X.; Jin, H.; Wang, S.; Deng, Z.; Zhong, Y. VulDeePecker: A Deep Learning-Based System for Vulnerability Detection. In Proceedings of the 2018 Network and Distributed System Security Symposium, San Diego, CA, USA, 18–21 February 2018; Internet Society: Reston, VA, USA, 2018. [CrossRef]
110. Keras-Team/Keras. Available online: <https://github.com/keras-team/keras> (accessed on 1 June 2024).
111. Chiu, J.P.C.; Nichols, E. Named Entity Recognition with Bidirectional LSTM-CNNs. *Trans. Assoc. Comput. Linguist.* **2016**, *4*, 357–370. [CrossRef]
112. Sun, P.; Yang, X.; Zhao, X.; Wang, Z. An Overview of Named Entity Recognition. In Proceedings of the 2018 International Conference on Asian Language Processing (IALP), Bandung, Indonesia, 15–17 November 2018; IEEE: Piscataway, NJ, USA, 2018; pp. 273–278.
113. Hochreiter, S.; Schmidhuber, J. Long Short-Term Memory. *Neural Comput.* **1997**, *36*, 1735–1780. [CrossRef]
114. Huff, P.; McClanahan, K.; Le, T.; Li, Q. A Recommender System for Tracking Vulnerabilities. In Proceedings of the 16th International Conference on Availability, Reliability and Security, Vienna, Austria, 17 August 2021; ACM: New York, NY, USA, 2021; pp. 1–7.
115. spaCy · Industrial-Strength Natural Language Processing in Python. Available online: <https://spacy.io/> (accessed on 25 May 2024).
116. Rahutomo, F.; Kitasuka, T.; Aritsugi, M. Semantic Cosine Similarity. In Proceedings of the 7th International Student Conference on Advanced Science and Technology ICAST, Seoul, Republic of Korea, 29–30 October 2012.
117. Kwak, B.I.; Han, M.L.; Kim, H.K. Cosine Similarity Based Anomaly Detection Methodology for the CAN Bus. *Expert Syst. Appl.* **2021**, *166*, 114066. [CrossRef]
118. Mihoub, A.; Fredj, O.B.; Cheikhrouhou, O.; Derhab, A.; Krichen, M. Denial of Service Attack Detection and Mitigation for Internet of Things Using Looking-Back-Enabled Machine Learning Techniques. *Comput. Electr. Eng.* **2022**, *98*, 107716. [CrossRef]

119. Qu, Y.; Uddin, M.P.; Gan, C.; Xiang, Y.; Gao, L.; Yearwood, J. Blockchain-Enabled Federated Learning: A Survey. *ACM Comput. Surv.* **2023**, *55*, 1–35. [\[CrossRef\]](#)
120. Torres, C.F.; Iannillo, A.K.; Gervais, A.; State, R. The Eye of Horus: Spotting and Analyzing Attacks on Ethereum Smart Contracts. In Proceedings of the International Conference on Financial Cryptography and Data Security, Virtual, 15 January 2021. [\[CrossRef\]](#)
121. Sun, X.; Tu, L.; Zhang, J.; Cai, J.; Li, B.; Wang, Y. ASSBert: Active and Semi-Supervised Bert for Smart Contract Vulnerability Detection. *J. Inf. Secur. Appl.* **2023**, *73*, 103423. [\[CrossRef\]](#)
122. Huang, S.; Jin, R.; Zhou, Z. Active Learning by Querying Informative and Representative Examples. *Adv. Neural Inf. Process. Syst.* **2010**, *23*. [\[CrossRef\]](#) [\[PubMed\]](#)
123. Taherkhani, F.; Kazemi, H.; Nasrabadi, N.M. Matrix Completion for Graph-Based Deep Semi-Supervised Learning. In Proceedings of the Thirty-Third AAAI Conference on Artificial Intelligence, Honolulu, HI, USA, 27 January–1 February 2019. [\[CrossRef\]](#)
124. Arazo, E.; Ortego, D.; Albert, P.; O'Connor, N.E.; McGuinness, K. Pseudo-Labeling and Confirmation Bias in Deep Semi-Supervised Learning. In Proceedings of the 2020 International Joint Conference on Neural Networks (IJCNN), Glasgow, UK, July 2020; IEEE: Piscataway, NJ, USA, 2020; pp. 1–8.
125. Yalniz, I.Z.; Jégou, H.; Chen, K.; Paluri, M.; Mahajan, D. Billion-Scale Semi-Supervised Learning for Image Classification. *arXiv* **2019**, arXiv:1905.00546.
126. Devlin, J.; Chang, M.-W.; Lee, K.; Toutanova, K. BERT: Pre-Training of Deep Bidirectional Transformers for Language Understanding. *arXiv* **2018**, arXiv:1810.04805.
127. Wen, X.-C.; Wang, X.; Chen, Y.; Hu, R.; Lo, D.; Gao, C. VulEval: Towards Repository-Level Evaluation of Software Vulnerability Detection. *arXiv* **2024**, arXiv:2404.15596.
128. Hou, X.; Zhao, Y.; Liu, Y.; Yang, Z.; Wang, K.; Li, L.; Luo, X.; Lo, D.; Grundy, J.; Wang, H. Large Language Models for Software Engineering: A Systematic Literature Review. *arXiv* **2023**, arXiv:2308.10620v6. [\[CrossRef\]](#)
129. Touvron, H.; Lavril, T.; Izacard, G.; Martinet, X.; Lachaux, M.-A.; Lacroix, T.; Rozière, B.; Goyal, N.; Hambro, E.; Azhar, F.; et al. LLaMA: Open and Efficient Foundation Language Models. *arXiv* **2023**, arXiv:2302.13971.
130. Rozière, B.; Gehring, J.; Gloeckle, F.; Sootla, S.; Gat, I.; Tan, X.E.; Adi, Y.; Liu, J.; Sauvestre, R.; Remez, T.; et al. Code Llama: Open Foundation Models for Code. *arXiv* **2023**, arXiv:2308.12950.
131. ChatGPT. Available online: <https://chatgpt.com> (accessed on 2 June 2024).
132. Tariq, U. Combatting Ransomware in ZephyrOS-Activated Industrial IoT Environments. *Heliyon* **2024**, *10*, e29917. [\[CrossRef\]](#) [\[PubMed\]](#)
133. Koroniotis, N.; Moustafa, N.; Sitnikova, E.; Turnbull, B. Towards the Development of Realistic Botnet Dataset in the Internet of Things for Network Forensic Analytics: Bot-IoT Dataset. *Future Gener. Comput. Syst.* **2019**, *100*, 779–796. [\[CrossRef\]](#)
134. Durieux, T.; Ferreira, J.F.; Abreu, R.; Cruz, P. Empirical Review of Automated Analysis Tools on 47,587 Ethereum Smart Contracts. In Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering, Seoul, Republic of Korea, 27 June 2020; ACM: New York, NY, USA, 2020; pp. 530–541.
135. SoliAudit VA Dataset. Available online: https://docs.google.com/spreadsheets/u/1/d/17QxTGZA7xNifAV8bQ2A2dJWRRHcmPp3QgPNxwptT9Zw/edit?pli=1&usp=embed_facebook (accessed on 29 May 2024).
136. Ghaleb, A.; Pattabiraman, K. How Effective Are Smart Contract Analysis Tools? Evaluating Smart Contract Static Analysis Tools Using Bug Injection. In Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis, Virtual, 18 July 2020; ACM: New York, NY, USA, 2020; pp. 415–427.
137. Abdullahi, M.; Baashar, Y.; Alhussian, H.; Alwadain, A.; Aziz, N.; Capretz, L.F.; Abdulkadir, S.J. Detecting Cybersecurity Attacks in Internet of Things Using Artificial Intelligence Methods: A Systematic Literature Review. *Electronics* **2022**, *11*, 198. [\[CrossRef\]](#)
138. Amoo, O.O.; Osasona, F.; Atadoga, A.; Ayinla, B.S.; Farayola, O.A.; Abrahams, T.O. Cybersecurity Threats in the Age of IoT: A Review of Protective Measures. *Int. J. Sci. Res. Arch.* **2024**, *11*, 1304–1310. [\[CrossRef\]](#)
139. Ahmad, W.; Rasool, A.; Javed, A.R.; Baker, T.; Jalil, Z. Cyber Security in IoT-Based Cloud Computing: A Comprehensive Survey. *Electronics* **2021**, *11*, 16. [\[CrossRef\]](#)
140. Buda, M.; Maki, A.; Mazurowski, M.A. A Systematic Study of the Class Imbalance Problem in Convolutional Neural Networks. *Neural Netw.* **2018**, *106*, 249–259. [\[CrossRef\]](#)
141. Senanayake, J.; Kalutarage, H.; Al-Kadri, M.O.; Piras, L.; Petrovski, A. Labelled Vulnerability Dataset on Android Source Code (LVDAndro) to Develop AI-Based Code Vulnerability Detection Models. In Proceedings of the 20th International Conference on Security and Cryptography, Rome, Italy, 10–12 July 2023; SCITEPRESS—Science and Technology Publications: Setúbal, Portugal, 2023; pp. 659–666.
142. Rezaeibagha, F.; Mu, Y.; Huang, K.; Chen, L. Secure and Efficient Data Aggregation for IoT Monitoring Systems. *IEEE Internet Things J.* **2021**, *8*, 8056–8063. [\[CrossRef\]](#)
143. Pinconschi, E.; Reis, S.; Zhang, C.; Abreu, R.; Erdogmus, H.; Păsăreanu, C.S.; Jia, L. Tenet: A Flexible Framework for Machine-Learning-Based Vulnerability Detection. In Proceedings of the 2023 IEEE/ACM 2nd International Conference on AI Engineering—Software Engineering for AI (CAIN), Melbourne, Australia, 15–16 May 2023; IEEE: Piscataway, NJ, USA, 2023; pp. 102–103.
144. Stellios, I.; Kotzanikolaou, P.; Psarakis, M. Advanced Persistent Threats and Zero-Day Exploits in Industrial Internet of Things. In *Security and Privacy Trends in the Industrial Internet of Things*; Alcaraz, C., Ed.; Advanced Sciences and Technologies for Security Applications; Springer International Publishing: Cham, Switzerland, 2019; pp. 47–68, ISBN 978-3-030-12329-1.

145. Singh, S.; Sharma, P.K.; Moon, S.Y.; Moon, D.; Park, J.H. A Comprehensive Study on APT Attacks and Countermeasures for Future Networks and Communications: Challenges and Solutions. *J. Supercomput.* **2019**, *75*, 4543–4574. [\[CrossRef\]](#)
146. Admass, W.S.; Munaye, Y.Y.; Diro, A.A. Cyber Security: State of the Art, Challenges and Future Directions. *Cyber Secur. Appl.* **2024**, *2*, 100031. [\[CrossRef\]](#)
147. Maglaras, L.; Janicke, H.; Ferrag, M.A. Cybersecurity of Critical Infrastructures: Challenges and Solutions. *Sensors* **2022**, *22*, 5105. [\[CrossRef\]](#)
148. Djenna, A.; Harous, S.; Saidouni, D.E. Internet of Things Meet Internet of Threats: New Concern Cyber Security Issues of Critical Cyber Infrastructure. *Appl. Sci.* **2021**, *11*, 4580. [\[CrossRef\]](#)
149. Soe, Y.N.; Feng, Y.; Santosa, P.I.; Hartanto, R.; Sakurai, K. Towards a Lightweight Detection System for Cyber Attacks in the IoT Environment Using Corresponding Features. *Electronics* **2020**, *9*, 144. [\[CrossRef\]](#)
150. Long, Z.; Yan, H.; Shen, G.; Zhang, X.; He, H.; Cheng, L. A Transformer-Based Network Intrusion Detection Approach for Cloud Security. *J. Cloud Comput.* **2024**, *13*, 5. [\[CrossRef\]](#)
151. Jameil, A.K.; Al-Raweshidy, H. AI-Enabled Healthcare and Enhanced Computational Resource Management With Digital Twins Into Task Offloading Strategies. *IEEE Access* **2024**, *12*, 90353–90370. [\[CrossRef\]](#)
152. Li, T.; Sahu, A.K.; Talwalkar, A.; Smith, V. Federated Learning: Challenges, Methods, and Future Directions. *IEEE Signal Process. Mag.* **2020**, *37*, 50–60. [\[CrossRef\]](#)
153. Okoli, U.I.; Obi, O.C.; Adewusi, A.O.; Abrahams, T.O. Machine Learning in Cybersecurity: A Review of Threat Detection and Defense Mechanisms. *World J. Adv. Res. Rev.* **2024**, *21*, 2286–2295. [\[CrossRef\]](#)
154. Salem, A.H.; Azzam, S.M.; Emam, O.E.; Abohany, A.A. Advancing Cybersecurity: A Comprehensive Review of AI-Driven Detection Techniques. *J. Big Data* **2024**, *11*, 105. [\[CrossRef\]](#)
155. Denz, R.; Taylor, S. A Survey on Securing the Virtual Cloud. *J. Cloud Comput. Adv. Syst. Appl.* **2013**, *2*, 17. [\[CrossRef\]](#)
156. Guo, W.; Fang, Y.; Huang, C.; Ou, H.; Lin, C.; Guo, Y. HyVulDect: A Hybrid Semantic Vulnerability Mining System Based on Graph Neural Network. *Comput. Secur.* **2022**, *121*, 102823. [\[CrossRef\]](#)
157. Taghavi, S.M.; Feyzi, F. Using Large Language Models to Better Detect and Handle Software Vulnerabilities and Cyber Security Threats, CC BY 4.0 License. 2024. Available online: https://www.researchgate.net/publication/380772943_Using_Large_Language_Models_to_Better_Detect_and_Handle_Software_Vulnerabilities_and_Cyber_Security_Threats (accessed on 31 July 2024). [\[CrossRef\]](#)
158. Dokeroglu, T.; Sevinc, E.; Kucukyilmaz, T.; Cosar, A. A Survey on New Generation Metaheuristic Algorithms. *Comput. Ind. Eng.* **2019**, *137*, 106040. [\[CrossRef\]](#)
159. Rajwar, K.; Deep, K.; Das, S. An Exhaustive Review of the Metaheuristic Algorithms for Search and Optimization: Taxonomy, Applications, and Open Challenges. *Artif. Intell. Rev.* **2023**, *56*, 13187–13257. [\[CrossRef\]](#) [\[PubMed\]](#)
160. Nong, Y.; Sharma, R.; Hamou-Lhadj, A.; Luo, X.; Cai, H. Open Science in Software Engineering: A Study on Deep Learning-Based Vulnerability Detection. *IEEE Trans. Softw. Eng.* **2023**, *49*, 1983–2005. [\[CrossRef\]](#)
161. Chen, Y.; Ding, Z.; Alowain, L.; Chen, X.; Wagner, D. DiverseVul: A New Vulnerable Source Code Dataset for Deep Learning Based Vulnerability Detection. In Proceedings of the 26th International Symposium on Research in Attacks, Intrusions and Defenses, Hong Kong, China, 16 October 2023; ACM: New York, NY, USA, 2023; pp. 654–668.
162. Yang, X.; Wang, S.; Li, Y.; Wang, S. Does Data Sampling Improve Deep Learning-Based Vulnerability Detection? Yeas! And Nays! In Proceedings of the 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE), Melbourne, Australia, 14–20 May 2023; IEEE: Piscataway, NJ, USA, 2023; pp. 2287–2298.
163. Nie, X.; Li, N.; Wang, K.; Wang, S.; Luo, X.; Wang, H. Understanding and Tackling Label Errors in Deep Learning-Based Vulnerability Detection (Experience Paper). In Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis, Seattle, WA, USA, 12 July 2023; ACM: New York, NY, USA, 2023; pp. 52–63.
164. Tang, W.; Tang, M.; Ban, M.; Zhao, Z.; Feng, M. CSGVD: A Deep Learning Approach Combining Sequence and Graph Embedding for Source Code Vulnerability Detection. *J. Syst. Softw.* **2023**, *199*, 111623. [\[CrossRef\]](#)
165. Liu, Z.; Jiang, M.; Zhang, S.; Zhang, J.; Liu, Y. A Smart Contract Vulnerability Detection Mechanism Based on Deep Learning and Expert Rules. *IEEE Access* **2023**, *11*, 77990–77999. [\[CrossRef\]](#)
166. Yuan, B.; Lu, Y.; Fang, Y.; Wu, Y.; Zou, D.; Li, Z.; Li, Z.; Jin, H. Enhancing Deep Learning-Based Vulnerability Detection by Building Behavior Graph Model. In Proceedings of the 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE), Melbourne, Australia, 14–20 May 2023; IEEE: Piscataway, NJ, USA, 2023; pp. 2262–2274.
167. Harzevili, N.S.; Belle, A.B.; Wang, J.; Wang, S.; Ming, Z.; Nagappan, N. A Survey on Automated Software Vulnerability Detection Using Machine Learning and Deep Learning. *arXiv*, 2023. [\[CrossRef\]](#)
168. Steenhoek, B.; Rahman, M.M.; Jiles, R.; Le, W. An Empirical Study of Deep Learning Models for Vulnerability Detection. In Proceedings of the 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE), Melbourne, Australia, 17–19 May 2023. [\[CrossRef\]](#)
169. Yuan, Y.; Xie, T. SVChecker: A Deep Learning-Based System for Smart Contract Vulnerability Detection. In Proceedings of the International Conference on Computer Application and Information Security (ICCAIS 2021), Wuhan, China, 25 May 2022; Lu, Y., Cheng, C., Eds.; SPIE: Bellingham, WA, USA, 2022; p. 99.
170. Hussan, B.K.; Rashid, Z.N.; Zeebaree, S.R.M.; Zebari, R.R. Optimal Deep Belief Network Enabled Vulnerability Detection on Smart Environment. *J. Smart Internet Things* **2022**, *2022*, 146–162. [\[CrossRef\]](#)

171. Russell, R.L.; Kim, L.; Hamilton, L.H.; Lazovich, T.; Harer, J.A.; Ozdemir, O.; Ellingwood, P.M.; McConley, M.W. Automated Vulnerability Detection in Source Code Using Deep Representation Learning. In Proceedings of the 2018 17th IEEE International Conference on Machine Learning and Applications (ICMLA), Orlando, FL, USA, 17–20 December 2018. [\[CrossRef\]](#)
172. Zhou, Y.; Sharma, A. Automated Identification of Security Issues from Commit Messages and Bug Reports. In Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering, Paderborn, Germany, 21 August 2017; ACM: New York, NY, USA, 2017; pp. 914–919.
173. Russo, E.R.; Di Sorbo, A.; Visaggio, C.A.; Canfora, G. Summarizing Vulnerabilities' Descriptions to Support Experts during Vulnerability Assessment Activities. *J. Syst. Softw.* **2019**, *156*, 84–99. [\[CrossRef\]](#)
174. Li, Y.; Wang, S.; Nguyen, T.N. Vulnerability Detection with Fine-Grained Interpretations. In Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, Athens, Greece, 20 August 2021; ACM: New York, NY, USA, 2021; pp. 292–303.
175. Li, D.; Liu, Y.; Huang, J. Assessment of Software Vulnerability Contributing Factors by Model-Agnostic Explainable AI. *Mach. Learn. Knowl. Extr.* **2024**, *6*, 1087–1113. [\[CrossRef\]](#)
176. Zhang, F.; Huff, P.; McClanahan, K.; Li, Q. A Machine Learning-Based Approach for Automated Vulnerability Remediation Analysis. In Proceedings of the 2020 IEEE Conference on Communications and Network Security (CNS), Avignon, France, 29 June–1 July 2020; IEEE: Piscataway, NJ, USA, 2020; pp. 1–9.
177. Hassan, M.d.M.; Ahmad, R.B.; Ghosh, T. SQL Injection Vulnerability Detection Using Deep Learning: A Feature-Based Approach. *Indones. J. Electr. Eng. Inform. IJEEI* **2021**, *9*, 702–718. [\[CrossRef\]](#)
178. Hu, L.; Chang, J.; Chen, Z.; Hou, B. Web Application Vulnerability Detection Method Based on Machine Learning. *J. Phys. Conf. Ser.* **2021**, *1827*, 012061. [\[CrossRef\]](#)
179. Cao, Y.; Zhang, L.; Zhao, X.; Jin, K.; Chen, Z. An Intrusion Detection Method for Industrial Control System Based on Machine Learning. *Information* **2022**, *13*, 322. [\[CrossRef\]](#)
180. Hulayyil, S.B.; Li, S.; Xu, L. Machine-Learning-Based Vulnerability Detection and Classification in Internet of Things Device Security. *Electronics* **2023**, *12*, 3927. [\[CrossRef\]](#)
181. Shaukat, K.; Luo, S.; Chen, S.; Liu, D. Cyber Threat Detection Using Machine Learning Techniques: A Performance Evaluation Perspective. In Proceedings of the 2020 International Conference on Cyber Warfare and Security (ICWS), Islamabad, Pakistan, 20 October 2020; IEEE: Piscataway, NJ, USA, 2020; pp. 1–6.
182. Abdusalomov, A.; Kilichev, D.; Nasimov, R.; Rakhmatullayev, I.; Im Cho, Y. Optimizing Smart Home Intrusion Detection with Harmony-Enhanced Extra Trees. *IEEE Access* **2024**, *12*, 117761–117786. [\[CrossRef\]](#)
183. Gawand, S.P.; Kumar, M.S. A Comparative Study of Cyber Attack Detection & Prediction Using Machine Learning Algorithms. *Preprint* **2023**. [\[CrossRef\]](#)
184. Azhagiri, M.; Rajesh, A.; Karthik, S.; Raja, K. An Intrusion Detection System Using Ranked Feature Bagging. *Int. J. Inf. Technol.* **2023**, *16*, 1213–1219. [\[CrossRef\]](#)
185. Rodriguez, E.; Otero, B.; Gutierrez, N.; Canal, R. A Survey of Deep Learning Techniques for Cybersecurity in Mobile Networks. *IEEE Commun. Surv. Tutor.* **2021**, *23*, 1920–1955. [\[CrossRef\]](#)
186. Boi, B.; Esposito, C.; Lee, S. VulnHunt-GPT: A Smart Contract Vulnerabilities Detector Based on OpenAI chatGPT. In Proceedings of the 39th ACM/SIGAPP Symposium on Applied Computing, Avila, Spain, 8 April 2024; ACM: New York, NY, USA, 2024; pp. 1517–1524.
187. Ding, Y.; Fu, Y.; Ibrahim, O.; Sitawarin, C.; Chen, X.; Alomair, B.; Wagner, D.; Ray, B.; Chen, Y. Vulnerability Detection with Code Language Models: How Far Are We? *arXiv* **2024**. [\[CrossRef\]](#)
188. Zhou, X.; Cao, S.; Sun, X.; Lo, D. Large Language Model for Vulnerability Detection and Repair: Literature Review and the Road Ahead. *arXiv* **2024**, arXiv:2404.02525.
189. Xu, H.; Wang, S.; Li, N.; Wang, K.; Zhao, Y.; Chen, K.; Yu, T.; Liu, Y.; Wang, H. Large Language Models for Cyber Security: A Systematic Literature Review. *arXiv* **2024**, arXiv:2405.04760.
190. Yin, X.; Ni, C.; Wang, S. Multitask-Based Evaluation of Open-Source LLM on Software Vulnerability. *arXiv* **2024**, arXiv:2404.02056.
191. Steenhoek, B.; Rahman, M.M.; Roy, M.K.; Alam, M.S.; Barr, E.T.; Le, W. A Comprehensive Study of the Capabilities of Large Language Models for Vulnerability Detection. *arXiv* **2024**, arXiv:2403.17218.
192. Li, Z.; Dutta, S.; Naik, M. LLM-Assisted Static Analysis for Detecting Security Vulnerabilities. *arXiv* **2024**, arXiv:2405.17238.
193. Fang, R.; Bindu, R.; Gupta, A.; Kang, D. LLM Agents Can Autonomously Exploit One-Day Vulnerabilities. *arXiv* **2024**, arXiv:2404.08144.
194. Zhou, X.; Zhang, T.; Lo, D. Large Language Model for Vulnerability Detection: Emerging Results and Future Directions. In Proceedings of the 2024 ACM/IEEE 44th International Conference on Software Engineering: New Ideas and Emerging Results, Lisbon, Portugal, 14 April 2024; ACM: New York, NY, USA, 2024; pp. 47–51.
195. Sun, Y.; Wu, D.; Xue, Y.; Liu, H.; Ma, W.; Zhang, L.; Shi, M.; Liu, Y. LLM4Vuln: A Unified Evaluation Framework for Decoupling and Enhancing LLMs' Vulnerability Reasoning. *arXiv* **2024**, arXiv:2401.16185.
196. Tóth, R.; Bisztray, T.; Erdodi, L. LLMs in Web Development: Evaluating LLM-Generated PHP Code Unveiling Vulnerabilities and Limitations. In Proceedings of the International Conference on Computer Safety, Reliability, and Security, Florence, Italy, 17–20 September 2024. [\[CrossRef\]](#)

197. Ullah, S.; Han, M.; Pearce, S.P.H.; Coskun, A.; Stringhini, G. LLMs Cannot Reliably Identify and Reason About Security Vulnerabilities (Yet?): A Comprehensive Evaluation, Framework, and Benchmarks. In Proceedings of the IEEE Symposium on Security and Privacy, Francisco, CA, USA, 20–22 May 2024. [\[CrossRef\]](#)
198. Yao, Y.; Duan, J.; Xu, K.; Cai, Y.; Sun, Z.; Zhang, Y. A Survey on Large Language Model (LLM) Security and Privacy: The Good, The Bad, and The Ugly. *High-Confid. Comput.* **2024**, *4*, 100211. [\[CrossRef\]](#)
199. Mathews, N.S.; Brus, Y.; Aafer, Y.; Nagappan, M.; McIntosh, S. LLbezpeky: Leveraging Large Language Models for Vulnerability Detection. *arXiv* **2024**, arXiv:2401.01269.
200. Shestov, A.; Levichev, R.; Mussabayev, R.; Maslov, E.; Cheshkov, A.; Zadorozhny, P. Finetuning Large Language Models for Vulnerability Detection. *arXiv* **2024**, arXiv:2401.17010.
201. Sun, Y.; Wu, D.; Xue, Y.; Liu, H.; Wang, H.; Xu, Z.; Xie, X.; Liu, Y. GPTScan: Detecting Logic Vulnerabilities in Smart Contracts by Combining GPT with Program Analysis. In Proceedings of the IEEE/ACM 46th International Conference on Software Engineering, Lisbon, Portugal, 12 April 2024; ACM: New York, NY, USA, 2024; pp. 1–13.
202. Jones, A.; Omar, M. Codesentry: Revolutionizing Real-Time Software Vulnerability Detection With Optimized GPT Framework. *Land Forces Acad. Rev.* **2024**, *29*, 98–107. [\[CrossRef\]](#)
203. Ferrag, M.A.; Alwahedi, F.; Battah, A.; Cherif, B.; Mechri, A.; Tihanyi, N. Generative AI and Large Language Models for Cyber Security: All Insights You Need. *arXiv* **2024**, arXiv:2405.12750.
204. Manjunatha, A.; Kota, K.; Babu, A.S. CVE Severity Prediction from Vulnerability Description—A Deep Learning Approach. *Procedia Comput. Sci.* **2024**, *235*, 3105–3117. [\[CrossRef\]](#)
205. Rawte, V.; Tonmoy, S.M.T.I.; Rajbangshi, K.; Nag, S.; Chadha, A.; Sheth, A.P.; Das, A. FACTOID: FACTual enTailment fOr hallucination Detection. *arXiv* **2024**, arXiv:2403.19113.
206. Agrawal, P.; Abutarboush, H.F.; Ganesh, T.; Mohamed, A.W. Metaheuristic Algorithms on Feature Selection: A Survey of One Decade of Research (2009–2019). *IEEE Access* **2021**, *9*, 26766–26791. [\[CrossRef\]](#)
207. Zeinalpour, A.; McElroy, C.P. Comparing Metaheuristic Search Techniques in Addressing the Effectiveness of Clustering-Based DDoS Attack Detection Methods. *Electronics* **2024**, *13*, 899. [\[CrossRef\]](#)
208. Thomas, M.; Meshram, B.B. DoS Attack Detection Using Aquila Deer Hunting Optimization Enabled Deep Belief Network. *Int. J. Web Inf. Syst.* **2024**, *20*, 66–87. [\[CrossRef\]](#)
209. Syed, R. Cybersecurity Vulnerability Management: A Conceptual Ontology and Cyber Intelligence Alert System. *Inf. Manag.* **2020**, *57*, 103334. [\[CrossRef\]](#)
210. Jia, Y.; Qi, Y.; Shang, H.; Jiang, R.; Li, A. A Practical Approach to Constructing a Knowledge Graph for Cybersecurity. *Engineering* **2018**, *4*, 53–60. [\[CrossRef\]](#)
211. Martínez, S.; Cosentino, V.; Cabot, J. Model-Based Analysis of Java EE Web Security Misconfigurations. *Comput. Lang. Syst. Struct.* **2017**, *49*, 36–61. [\[CrossRef\]](#)
212. Seidl, C.; Winkelmann, T.; Schaefer, I. A Software Product Line of Feature Modeling Notations and Cross-Tree Constraint Languages. 2016, pp. 157–172. Available online: <https://dl.gi.de/items/758130c0-32b3-485e-8d9d-04e1e1f94a8f> (accessed on 21 July 2024).
213. Sawyer, P.; Mazo, R.; Diaz, D.; Salinesi, C.; Hughes, D. Using Constraint Programming to Manage Configurations in Self-Adaptive Systems. *Computer* **2012**, *45*, 56–63. [\[CrossRef\]](#)
214. Felfernig, A.; Walter, R.; Galindo, J.A.; Benavides, D.; Erdeniz, S.P.; Atas, M.; Reiterer, S. Anytime Diagnosis for Reconfiguration. *J. Intell. Inf. Syst.* **2018**, *51*, 161–182. [\[CrossRef\]](#)
215. Varela-Vaca, Á.J.; Galindo, J.A.; Ramos-Gutiérrez, B.; Gómez-López, M.T.; Benavides, D. Process Mining to Unleash Variability Management: Discovering Configuration Workflows Using Logs. In Proceedings of the 23rd International Systems and Software Product Line Conference, Paris, France, 9 September 2019; ACM: New York, NY, USA, 2019; Volume A, pp. 265–276.
216. Costa, G.; Merlo, A.; Verderame, L.; Armando, A. Automatic Security Verification of Mobile App Configurations. *Future Gener. Comput. Syst.* **2018**, *80*, 519–536. [\[CrossRef\]](#)
217. Murthy, P.V.R.; Shilpa, R.G. Vulnerability Coverage Criteria for Security Testing of Web Applications. In Proceedings of the 2018 International Conference on Advances in Computing, Communications and Informatics (ICACCI), Bangalore, India, 19–22 September 2018; IEEE: Piscataway, NJ, USA, 2018; pp. 489–494.
218. Xiong, W.; Lagerström, R. Threat Modeling—A Systematic Literature Review. *Comput. Secur.* **2019**, *84*, 53–69. [\[CrossRef\]](#)
219. Thüm, T.; Kästner, C.; Benduhn, F.; Meinicke, J.; Saake, G.; Leich, T. FeatureIDE: An Extensible Framework for Feature-Oriented Software Development. *Sci. Comput. Program.* **2014**, *79*, 70–85. [\[CrossRef\]](#)
220. Blanco, C.; Rosado, D.G.; Varela-Vaca, Á.J.; Gómez-López, M.T.; Fernández-Medina, E. Onto-CARMEN: Ontology-Driven Approach for Cyber-Physical System Security Requirements Meta-Modelling and Reasoning. *Internet Things* **2023**, *24*, 100989. [\[CrossRef\]](#)
221. Hitesh; Kumari, A.C. Feature Selection Optimization in SPL Using Genetic Algorithm. *Procedia Comput. Sci.* **2018**, *132*, 1477–1486. [\[CrossRef\]](#)
222. Zahoor Chohan, A.; Bibi, A.; Hafeez Motla, Y. Optimized Software Product Line Architecture and Feature Modeling in Improvement of SPL. In Proceedings of the 2017 International Conference on Frontiers of Information Technology (FIT), Islamabad, Pakistan, 18–20 December 2017; IEEE: Piscataway, NJ, USA, 2017; pp. 167–172.

223. Zou, D.; Wang, S.; Xu, S.; Li, Z.; Jin, H. μ VulDeePecker: A Deep Learning-Based System for Multiclass Vulnerability Detection. *IEEE Trans. Dependable Secur. Comput.* **2019**, *18*, 2224–2236. [\[CrossRef\]](#)
224. Zhang, J.; Liu, Z.; Hu, X.; Xia, X.; Li, S. Vulnerability Detection by Learning From Syntax-Based Execution Paths of Code. *IEEE Trans. Softw. Eng.* **2023**, *49*, 4196–4212. [\[CrossRef\]](#)
225. Kreyßig, B.; Bartel, A. Analyzing Prerequisites of Known Deserialization Vulnerabilities on Java Applications. In Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering, Salerno, Italy, 18–21 June 2024. [\[CrossRef\]](#)
226. Aladics, T.; Hegedűs, P.; Ferenc, R. An AST-Based Code Change Representation and Its Performance in Just-in-Time Vulnerability Prediction. In Proceedings of the International Conference on Software Technologies, Rome, Italy, 10–12 July 2023. [\[CrossRef\]](#)
227. Wan, T.; Lu, L.; Xu, H.; Zou, Q. Software Vulnerability Detection via Doc2vec via Path Representation. In Proceedings of the 2023 IEEE 23rd International Conference on Software Quality, Reliability, and Security Companion (QRS-C), Chiang Mai, Thailand, 22–26 October 2023; IEEE: Piscataway, NJ, USA, 2023. [\[CrossRef\]](#)
228. Liu, R.; Wang, Y.; Xu, H.; Liu, B.; Sun, J.; Guo, Z.; Ma, W. Source Code Vulnerability Detection: Combining Code Language Models and Code Property Graphs. *arXiv* **2024**, arXiv:2404.14719.
229. Zhao, C.; Tu, T.; Wang, C.; Qin, S. VulPathsFinder: A Static Method for Finding Vulnerable Paths in PHP Applications Based on CPG. *Appl. Sci.* **2023**, *13*, 9240. [\[CrossRef\]](#)
230. Wu, P.; Yin, L.; Du, X.; Jia, L.; Dong, W. Graph-Based Vulnerability Detection via Extracting Features from Sliced Code. In Proceedings of the 2020 IEEE 20th International Conference on Software Quality, Reliability and Security Companion (QRS-C), Macau, China, 11–14 December 2020; IEEE: Piscataway, NJ, USA, 2020; pp. 38–45.
231. Wu, Y.; Lu, J.; Zhang, Y.; Jin, S. Vulnerability Detection in C/C++ Source Code with Graph Representation Learning. In Proceedings of the 2021 IEEE 11th Annual Computing and Communication Workshop and Conference (CCWC), Virtual, 27–30 January 2021; IEEE: Piscataway, NJ, USA, 2021; pp. 1519–1524.
232. Zhang, C.; Xin, Y. Static Vulnerability Detection Based on Class Separation. *J. Syst. Softw.* **2023**, *206*, 111832. [\[CrossRef\]](#)
233. Şahin, C.B. Semantic-Based Vulnerability Detection by Functional Connectivity of Gated Graph Sequence Neural Networks. *Soft Comput.* **2023**, *27*, 5703–5719. [\[CrossRef\]](#)
234. Gong, K.; Song, X.; Wang, N.; Wang, C.; Zhu, H. SCGformer: Smart Contract Vulnerability Detection Based on Control Flow Graph and Transformer. *IET Blockchain* **2023**, *3*, 213–221. [\[CrossRef\]](#)
235. Yuan, X.; Lin, G.; Mei, H.; Tai, Y.; Zhang, J. Software Vulnerable Functions Discovery Based on Code Composite Feature. *J. Inf. Secur. Appl.* **2024**, *81*, 103718. [\[CrossRef\]](#)
236. Pradel, M.; Sen, K. DeepBugs: A Learning Approach to Name-Based Bug Detection. *Proc. ACM Program. Lang.* **2018**, *2*, 1–25. [\[CrossRef\]](#)
237. Javorník, M.; Komárková, J.; Husák, M. Decision Support for Mission-Centric Cyber Defence. In Proceedings of the 14th International Conference on Availability, Reliability and Security, Canterbury, UK, 26 August 2019; ACM: New York, NY, USA, 2019; pp. 1–8.
238. Husák, M.; Sadlek, L.; Špaček, S.; Laštovička, M.; Javorník, M.; Komárková, J. CRUSOE: A Toolset for Cyber Situational Awareness and Decision Support in Incident Handling. *Comput. Secur.* **2022**, *115*, 102609. [\[CrossRef\]](#)
239. Wagner, N.; Sahin, C.S.; Winterrose, M.; Riordan, J.; Pena, J.; Hanson, D.; Streilein, W.W. Towards Automated Cyber Decision Support: A Case Study on Network Segmentation for Security. In Proceedings of the 2016 IEEE Symposium Series on Computational Intelligence (SSCI), Athens, Greece, 6–9 December 2016; IEEE: Piscataway, NJ, USA, 2016; pp. 1–10.
240. Chen, X.; Jia, S.; Xiang, Y. A Review: Knowledge Reasoning over Knowledge Graph. *Expert Syst. Appl.* **2020**, *141*, 112948. [\[CrossRef\]](#)
241. Li, X.; Chen, J.; Lin, Z.; Zhang, L.; Wang, Z.; Zhou, M.; Xie, W. A Mining Approach to Obtain the Software Vulnerability Characteristics. In Proceedings of the 2017 Fifth International Conference on Advanced Cloud and Big Data (CBD), Shanghai, China, 13–16 August 2017; IEEE: Piscataway, NJ, USA, 2017; pp. 296–301.
242. Shi, Z.; Matyunin, N.; Graffi, K.; Starobinski, D. Uncovering Product Vulnerabilities with Threat Knowledge Graphs. In Proceedings of the 2022 IEEE Secure Development Conference (SecDev), Atlanta, GA, USA, 18–20 October 2022; IEEE: Piscataway, NJ, USA, 2022; pp. 84–90.
243. Wang, X.; He, X.; Cao, Y.; Liu, M.; Chua, T.-S. KGAT: Knowledge Graph Attention Network for Recommendation. In Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, Anchorage, AK, USA, 25 July 2019; pp. 950–958.
244. Allamanis, M.; Brockschmidt, M.; Khademi, M. Learning to Represent Programs with Graphs. *arXiv* **2017**, arXiv:1711.00740.
245. Cheng, X.; Wang, H.; Hua, J.; Xu, G.; Sui, Y. DeepWukong: Statically Detecting Software Vulnerabilities Using Deep Graph Neural Network. *ACM Trans. Softw. Eng. Methodol.* **2021**, *30*, 1–33. [\[CrossRef\]](#)
246. Kiran, S.R.A.; Rajper, S.; Shaikh, R.A.; Shah, I.A.; Danwar, S.H. Categorization of CVE Based on Vulnerability Software By Using Machine Learning Techniques. *Int. J. Adv. Trends Comput. Sci. Eng.* **2021**, *10*, 2637–2644. [\[CrossRef\]](#)
247. Li, Y.; Zhang, B. Detection of SQL Injection Attacks Based on Improved TFIDF Algorithm. *J. Phys. Conf. Ser.* **2019**, *1395*, 012013. [\[CrossRef\]](#)
248. Sun, H.; Cui, L.; Li, L.; Ding, Z.; Hao, Z.; Cui, J.; Liu, P. VDSimilar: Vulnerability Detection Based on Code Similarity of Vulnerabilities and Patches. *Comput. Secur.* **2021**, *110*, 102417. [\[CrossRef\]](#)

249. Kim, S.; Woo, S.; Lee, H.; Oh, H. VUDDY: A Scalable Approach for Vulnerable Code Clone Discovery. In Proceedings of the 2017 IEEE Symposium on Security and Privacy (SP), San Jose, CA, USA, 22–24 May 2017; IEEE: Piscataway, NJ, USA, 2017; pp. 595–614.
250. Hu, W.; Thing, V.L.L. CPE-Identifier: Automated CPE Identification and CVE Summaries Annotation with Deep Learning and NLP. *arXiv* **2024**, arXiv:2405.13568.
251. Kanakogi, K.; Washizaki, H.; Fukazawa, Y.; Ogata, S.; Okubo, T.; Kato, T.; Kanuka, H.; Hazeyama, A.; Yoshioka, N. Tracing CVE Vulnerability Information to CAPEC Attack Patterns Using Natural Language Processing Techniques. *Information* **2021**, *12*, 298. [\[CrossRef\]](#)
252. Reimers, N.; Gurevych, I. Sentence-BERT: Sentence Embeddings Using Siamese BERT-Networks. *arXiv* **2019**, arXiv:1908.10084.
253. O'Hare, J.; Macfarlane, R.; Lo, O. Identifying Vulnerabilities Using Internet-Wide Scanning Data. In Proceedings of the 2019 IEEE 12th International Conference on Global Security, Safety and Sustainability (ICGS3), London, UK, 16–18 January 2019; IEEE: Piscataway, NJ, USA, 2019; pp. 1–10.
254. Wang, X.; Sun, K.; Batcheller, A.; Jajodia, S. Detecting “0-Day” Vulnerability: An Empirical Study of Secret Security Patch in OSS. In Proceedings of the 2019 49th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN), Portland, OR, USA, 24–27 June 2019; IEEE: Piscataway, NJ, USA, 2019; pp. 485–492.
255. Takahashi, T.; Inoue, D. Generating Software Identifier Dictionaries from Vulnerability Database. In Proceedings of the 2016 14th Annual Conference on Privacy, Security and Trust (PST), Auckland, New Zealand, 12–14 December 2016; IEEE: Piscataway, NJ, USA, 2016; pp. 417–420.
256. Alfasi, D.; Shapira, T.; Barr, A.B. Unveiling Hidden Links Between Unseen Security Entities. *arXiv* **2024**, arXiv:2403.02014.
257. Chen, T.; Li, L.; Zhu, L.; Li, Z.; Liu, X.; Liang, G.; Wang, Q.; Xie, T. VulLibGen: Generating Names of Vulnerability-Affected Packages via a Large Language Model. In Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics, Bangkok, Thailand, 11–16 August 2024. [\[CrossRef\]](#)
258. Aghaei, E.; Al-Shaer, E.; Shadid, W.; Niu, X. Automated CVE Analysis for Threat Prioritization and Impact Prediction. *arXiv* **2023**, arXiv:2309.03040.
259. Blinowski, G.J.; Piotrowski, P. CVE Based Classification of Vulnerable IoT Systems. In *Theory and Applications of Dependable Computer Systems*; Zamojski, W., Mazurkiewicz, J., Sugier, J., Walkowiak, T., Kacprzyk, J., Eds.; Advances in Intelligent Systems and Computing; Springer International Publishing: Cham, Switzerland, 2020; Volume 1173, pp. 82–93, ISBN 978-3-030-48255-8.
260. Jiang, Y.; Atif, Y. Towards Automatic Discovery and Assessment of Vulnerability Severity in Cyber-Physical Systems. *Array* **2022**, *15*, 100209. [\[CrossRef\]](#)

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.